

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ярославский государственный университет имени П. Г. Демидова»

Кафедра дискретного анализа

Сдано на кафедру

«_____» _____ 2026 г.

Заведующий кафедрой,

д. ф.-м. н., доцент

_____ А. В. Николаев

Выпускная квалификационная работа

**Разработка игрового приложения
«Коридор» с модулем стратегического поведения
соперника**

по направлению

01.04.02 Прикладная математика и информатика

Научный руководитель

д. ф.-м. н., доцент

_____ А. В. Николаев

«_____» _____ 2026 г.

Студент группы ИВТ-21МО

_____ А. О. Карпунин

«_____» _____ 2026 г.

Ярославль, 2026

Реферат

Объем 67 с., 5 гл., 7 рис., 8 табл., 5 источников, 4 прил.

Ключевые слова: игры, Коридор, поиск кратчайшего пути, игровой искусственный интеллект, Минимакс, альфа-бета отсечение, Монте-Карло, функция оценки, обучение весов.

Объект исследования — логическая игра «Коридор» и алгоритмы автоматического расчета лучшего хода в ней.

Цель работы — разработка игрового приложения, с помощью которого пользователь сможет сыграть против одного из четырёх программных соперников, а также сравнить эти алгоритмы между собой.

Результат работы — приложение, реализующее правила игры «Коридор», с возможностью игры против компьютера или наблюдения за игрой алгоритмов друг против друга.

Содержание

Введение	5
1. Игра «Коридор»	6
1.1. Необходимые определения	6
1.2. Описание игры	6
2. Модуль стратегического поведения соперника	8
3. Алгоритмы	9
3.1. Общие функции и структура алгоритмов	9
3.2. Случайный алгоритм	13
3.3. Минимакс	14
3.4. Альфа-бета отсечения	16
3.5. Монте-Карло	18
3.5.1. Выбор	18
3.5.2. Расширение	19
3.5.3. Симуляция	19
3.5.4. Обратное распространение	20
3.5.5. Выбор лучшего хода	20
3.6. Сводные параметры сложности	21
4. Сравнение алгоритмов	22
4.1. Методика сравнения	22
4.2. Собираемые метрики	22
4.3. План эксперимента	22
4.4. Форма представления результатов	23
4.5. Сравнение весов функции оценки	24
4.6. Анализ результатов	24
5. Игровое приложение	25
5.1. Техническая реализация	25
5.2. Взаимодействие с приложением	27
Заключение	31
Список литературы	32
Приложение А. Реализация алгоритма MiniMax	33

Приложение Б. Реализация алгоритма AlphaBeta	40
Приложение В. Реализация алгоритма Monte Carlo	52
Приложение Г. Реализация обучения весов функции оценки	62

Введение

Коридор — настольная логическая игра для двух или четырёх игроков, выпущенная в 1997 году компанией Gigamic. Она была создана Мирко Маркези на основе более ранней игры «Блокада». Игра получила заметное распространение среди настольных логических игр, в частности, она входила в подборку Games 100 журнала *Games* [1].

Цель у игрока такова — необходимо довести свою фишку до противоположной стороны игрового поля быстрее, чем это сделает соперник. На каждом ходу игрок выбирает одно из следующих действий: продвинуть свою фишку к противоположной стороне поля или поставить стену, которая усложнит маршрут соперника. Количество таких стен ограничено, а правила игры запрещают полностью блокировать путь любого игрока к его целевой стороне поля.

Целью выпускной квалификационной работы является разработка игрового приложения «Коридор», включающего в себя реализацию правил игры, визуализацию игрового поля и модуль стратегического поведения, объединяющий четыре алгоритма программного соперника.

Для реализации цели работы были поставлены следующие задачи:

1. изучить правила игры «Коридор»,
2. реализовать алгоритм проверки корректности перемещения фишки,
3. реализовать алгоритм проверки корректности установки перегородки с проверкой достижимости целевых сторон поля,
4. реализовать алгоритм поиска кратчайшего пути до целевой стороны поля на основе обхода графа в ширину,
5. разработать функцию оценки игровой позиции с настраиваемыми весами и механизмом их коррекции по результатам партий,
6. реализовать четыре алгоритма программного соперника: случайный выбор хода, минимакс, альфа-бета отсечение и Монте-Карло,
7. провести сравнение алгоритмов в серии автоматических партий,
8. разработать интерфейс игрового приложения.

1. Игра «Коридор»

1.1. Необходимые определения

В формулировке правил игры и последующем описании алгоритмов встречается несколько понятий, которые нуждаются в пояснении.

Игровым полем называется квадратная доска размером $n \times n$ клеток, по которым перемещаются фишки игроков. В стандартном варианте игры $n = 9$, однако приложение также поддерживает поля 7×7 и 11×11 . Каждая из клеток нумеруется парой индексов (i, j) , где $0 \leq i, j \leq n - 1$.

Фишкой называется объект, который занимает одну клетку игрового поля и способен к перемещению на одну из смежных клеток.

Стеной называется элемент, который занимает пространство между клетками. Такая перегородка имеет длину две клетки и может быть расположена горизонтально, блокируя вертикальный проход между двумя парами клеток, или вертикально. Количество стен у каждого игрока ограничено: 8 для поля 7×7 , 10 для 9×9 , 12 для 11×11 .

Целевой линией игрока называется строка игрового поля, которая находится на противоположной стороне от его стартовой позиции. В начале партии игроки расположены на центрах противоположных сторон поля.

1.2. Описание игры

В начале партии фишки игроков расставляются на противоположных сторонах поля: первый игрок — в центре нижней строки, второй — в центре верхней. Они ходят по очереди, первенство определяется случайно. Цель каждого игрока — первым достичь любой клетки своей целевой линии.

За один ход игрок может выполнить только одно из двух действий [2]:

1. Перемещение по игровому полю. Фишка игрока передвигается на смежную клетку, если такой маневр не заблокирован стеной. Если соседняя клетка занята фишкой соперника и за ней нет ни перегородки, ни границы поля, фишка игрока может перепрыгнуть через соперника. Если прямой прыжок невозможен, допускается прыжок на клетку слева или справа от той, куда изначально предполагался прыжок. Варианты перемещения показаны на рис. 1.
2. Установка стены. Она допускается, если у игрока есть хотя бы одна перегородка в его запасе, она не пересекается с уже установленными стенами на игровом поле и после её установки у каждого из игроков остается хотя бы

один маршрут к своей целевой линии. Пример установки стены приведён на рис. 2.

Игра завершается, когда фишка одного из игроков достигает любой клетки целевой линии — этот игрок побеждает. Условие победы показано на рис. 3.

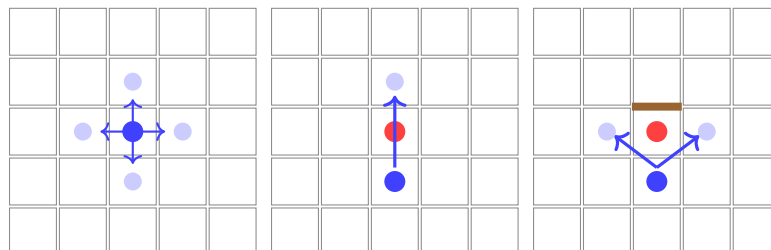


Рис. 1 — Варианты перемещения фишки

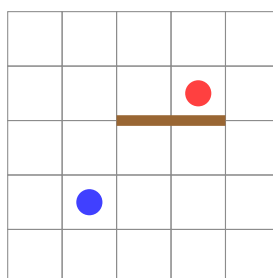


Рис. 2 — Пример установки стены

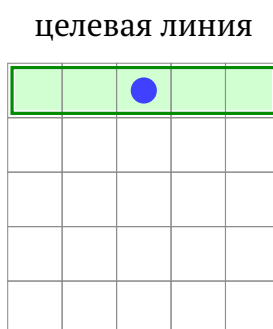


Рис. 3 — Условие победы

2. Модуль стратегического поведения соперника

Модуль стратегического поведения — это программный компонент, который отвечает за выбор хода соперника. Основываясь на полученных параметрах, а именно текущем состоянии игрового поля, количестве оставшихся стен у игроков, модуль возвращает лучший по собственной оценке ход.

В процессе разработки к модулю стратегического поведения соперника были выдвинуты следующие требования:

1. *Корректность*: выбранный алгоритмом ход не должен противоречить правилам игры.
2. *Завершённость*: алгоритм должен завершаться за конечное время. Все реализованные в приложении алгоритмы работают с ограничением по времени: каждый из них получает лимит в миллисекундах и прекращает поиск по его истечении. Этот устанавливаемый лимит зависит от сложности игры и выбранного размера игрового поля.
3. *Единое взаимодействие с выбранным алгоритмом*: все алгоритмы реализуют единый программный интерфейс `Algorithm` с методом получения лучшего хода. Благодаря этому переключение алгоритмов или добавление новых не требует вносить какие-либо изменения в остальные компоненты игрового приложения.
4. *Настраиваемая сложность*: каждый из алгоритмов поддерживает три уровня сложности. Выбранный уровень сложности влияет не только на ограничение по времени, но и на глубину поиска или число итераций алгоритма.

В рамках данной работы в модуле поведения соперника реализовано четыре алгоритма, каждый из которых может быть выбран в качестве оппонента для игры против пользователя. Случайный алгоритм равновероятно выбирает один из множества допустимых ходов. Минимакс перебирает дерево игровых состояний на ограниченную глубину без каких-либо отсечений. Альфа-бета расширяет минимакс, добавляя в изначальный алгоритм отсечения безперспективных ветвей дерева. Метод Монте-Карло оценивает позиции через многократные случайные симуляции игры из каждой из них.

3. Алгоритмы

3.1. Общие функции и структура алгоритмов

Каждый из разработанных алгоритмов реализуют единый программный интерфейс `Algorithm`, содержащий как абстрактные методы, тело которых определяется выбранным алгоритмом самостоятельно, так и конкретные реализации общих вспомогательных функций. Благодаря этому исключается дублирование кода и упрощается добавление нового алгоритма в приложение.

Генерация допустимых ходов

Метод `getMoves(board, playerId, wallsLeft)`, который применяется всеми алгоритмами в модуле стратегического поведения соперника, формирует полное множество допустимых ходов из текущего состояния из двух частей.

Первая часть — допустимые перемещения фишки, получаемые с помощью метода `board.getAvailableMoves(pos)`, который поочередно проверяет четыре направления перемещения с текущей клетки, а также обрабатывает прыжки через соперника, если такие возможны.

Вторая часть — допустимые варианты установки стен. Алгоритм формирует множество «кандидатных» клеток, ограничивая полный перебор стратегически значимой областью поля. Кандидатами являются клетки, которые расположены около кратчайшего пути соперника к его целевой строке поля. Для каждой такой клетки проверяются два возможных направления стены. Перегородка включается в список кандидатов, если она не имеет коллизий с уже расставленными перегородками на поле, и не отрезает ни одного из игроков от его целевой линии. Помимо прочего проверяется выполнение хотя бы одного условия:

1. стена увеличивает кратчайший путь соперника минимум на 2 хода
2. собственный путь удлиняется меньше, чем путь соперника

Итоговый список возможных стен сортируется по убыванию значения функции `wallImpact`.

Функция `wallImpact`

Функция `wallImpact(board, move, playerId, size)` оценивает стратегическую ценность конкретной установки стены. Применяв ход на копии доски, она вычисляет изменения длин путей участников игры до их целевых строк:

$$\text{wallImpact} = (d_p^{\text{after}} - d_p^{\text{before}}) \cdot 100 - \max(0, d_a^{\text{after}} - d_a^{\text{before}}) \cdot 45,$$

то есть каждый шаг, увеличивающий путь соперника, оценивается в 100 единиц, а каждый шаг, удлиняющий собственный путь, штрафует 45 единицами. Стена считается стратегически ценной, если $\text{wallImpact} > 0$.

Поиск кратчайшего пути

Метод `calculateShortestPath(board, start, targetRow)` основан на обходе графа в ширину от текущей позиции фишки на игровом поле до ближайшей клетки целевой строки.

Метод `getShortestPathCells` дополняет предыдущий восстановлением кратчайшего пути, поскольку при поиске такового хранит в истории посещенные клетки. Как уже описывалось ранее, он используется для поиска «кандидатных» клеток для установки стен.

Эндшпильные правила

Метод `findEndgameMove` вызывается в любом из алгоритмов в качестве первого шага перед запуском основного поиска лучшего хода. Если с текущей клетки поля фишку игрока можно переместить прямо его на целевую линию — этот ход немедленно возвращается без перебора других вариантов. Если такого «победного» хода нет, но соперник находится не более чем в двух ходах от победы, а лимит стен у алгоритма не исчерпан — ищется стена с максимальным значением `wallImpact`, чтобы замедлить игрока.

Предпочтение направления движения

С помощью метода `movementPreference` к оценке хода добавляется бонус или штраф в зависимости от направления перемещения и истории предыдущих ходов:

- ход в сторону целевой линии: +45 очков;
- ход назад: −120 очков;
- уменьшение длины пути до целевой линии на Δ клеток: $+35 \cdot \Delta$ очков;
- перемещение на позицию, которая встречалась в последних двух ходах: −500 очков;
- повтор более давнего перемещения: −180 очков.

Вся история перемещений хранится для каждого игрока отдельно и получается алгоритмом путем вызова `getRecentPositions`.

Функция оценки позиции

Функция оценки `evaluate()` является главным компонентом всех реализованных в игровом приложении алгоритмов. Благодаря ей каждому состоянию игры сопоставляется числовое значение, которое описывает его стратегическую выгоду для программного соперника. Оценка вычисляется как взвешенная сумма следующих параметров:

$$F(s) = \sum_{k=1}^7 \lambda_k \cdot f_k(s),$$

где λ_k — веса, а $f_k(s)$ — значения параметров. Параметры и их описание перечислены в таблице 3.1.

Таблица 3.1

Параметры оценочной функции

k	Признак $f_k(s)$	Описание
1	$d_p - d_a$	Разность между длинами кратчайших путей алгоритма и пользователя
2	mob_a	Количество допустимых ходов фишкой алгоритма — его мобильность
3	$-\text{mob}_p$	Отрицательное количество допустимых ходов пользователя
4	$w_a - w_p$	Преимущество алгоритма по числу оставшихся стен
5	$\text{prog}_a - \text{prog}_p$	Разность продвижений к целевым линиям алгоритма и пользователя
6	$e(d_a)$	Бонус за близость алгоритма к победе над игроком
7	$-e(d_p)$	Штраф за близость пользователя к победе над алгоритмом

Продвижение prog_a равно количеству строк игрового поля, которые прошла фишка алгоритма от стартовой позиции до текущей. Функция $e(d)$ может принимать такие значения:

$$e(d) = \begin{cases} 10, & d \leq 1, \\ 4, & d = 2, \\ 1, & d = 3, \\ 0, & d > 3. \end{cases}$$

Веса λ_k , соответствующие описанным параметрам, по умолчанию заданы следующим образом (класс `EvaluationWeights`):

$$\lambda_1 = 65, \quad \lambda_2 = 4, \quad \lambda_3 = 3, \quad \lambda_4 = 18, \quad \lambda_5 = 6, \quad \lambda_6 = 165, \quad \lambda_7 = 175.$$

Пример использования функции оценки приведён в таблице 3.2. Значения являются иллюстративными. Второй ход получает большую оценку, потому что сильнее увеличивает преимущество по кратчайшему пути, несмотря на меньшую мобильность.

Таблица 3.2

Пример сравнения двух ходов с помощью функции оценки

Вариант	$d_p - d_a$	mob_a	$-\text{mob}_p$	$w_a - w_p$	$\text{prog}_a - \text{prog}_p$	$e(d_a)$	$-e(d_p)$	$F(s)$
Ход фишкой вперёд	2	3	-2	0	1	0	0	262
Установка стены	4	2	-1	-1	0	0	0	474

Коррекция весов

Игровое приложение поддерживает механизм автоматической коррекции параметров функции оценки, анализируя результаты сыгранных партий (класс `EvaluationWeightsStore`). После завершения каждой из партий, в которых участвовал программный соперник, вычисляется оценка полезности ходов, которые предпринимал алгоритм.

В течение игры для каждого из ходов алгоритма формируется специальный объект `EvaluationLearningSample`, который хранит вектор параметров до хода и после него. После завершения партии для каждого образца вычисляется дельта параметров оценки $\Delta f_k = f_k^{\text{after}} - f_k^{\text{before}}$ и суммируется в градиент с учётом знака:

$$g_k = \sum_{\text{ходы алгоритма}} \text{reward} \cdot \Delta f_k, \quad \text{reward} = \begin{cases} +1, & \text{алгоритм победил,} \\ -1, & \text{алгоритм проиграл.} \end{cases}$$

Если $g_k > 0$, увеличение веса k -го параметра чаще встречалось в ходах победителя, поэтому вес увеличивается. Если $g_k < 0$, этот признак чаще участвовал в принятии решений, приводящих к проигрышу, поэтому соответствующий ему вес уменьшается. Если $g_k = 0$, вес не изменяется. Для начала необходимо вычислить предварительное значение обновленного веса:

$$\lambda'_k = \lambda_k + \text{sign}(g_k) \cdot \Delta \lambda.$$

Затем оно проверяется согласно допустимому диапазону:

$$\lambda_k^{\text{new}} = \begin{cases} \lambda_k^{\min}, & \lambda'_k < \lambda_k^{\min}, \\ \lambda_k^{\max}, & \lambda'_k > \lambda_k^{\max}, \\ \lambda'_k, & \lambda_k^{\min} \leq \lambda'_k \leq \lambda_k^{\max}. \end{cases}$$

Здесь $\Delta\lambda = 1$ — это шаг обучения, а $\lambda_k^{\min}$ и $\lambda_k^{\max}$ — минимально и максимально допустимые значения веса. Такие ограничения были наложены на веса параметров, чтобы после серии обучающих партий с алгоритмом отдельный параметр не стал чрезмерно большим, отрицательным или слишком малым и не начал полностью подавлять остальные признаки функции оценки. Актуальные веса параметров регулярно сохраняются в файл `data/evaluation-weights.properties` и используются для функции оценки в следующем запуске приложения, что обеспечивает накопление опыта между сессиями. На момент написания работы механизм коррекции предпринял 15 обновлений весов на 1388 обучающих образцах, а текущие сохранённые веса составляют:

$$\lambda_1 = 120, \quad \lambda_2 = 7, \quad \lambda_3 = 12, \quad \lambda_4 = 8, \quad \lambda_5 = 25, \quad \lambda_6 = 186, \quad \lambda_7 = 170.$$

Сравнение изначально выбранных и обновленных весов показывает, что механизм увеличил приоритет разности путей ($\lambda_1 : 65 \rightarrow 120$), штрафа за количество возможных ходов соперника ($\lambda_3 : 3 \rightarrow 12$), продвижения к целевой строке игрового поля ($\lambda_5 : 6 \rightarrow 25$) и собственного преимущества ($\lambda_6 : 165 \rightarrow 186$), одновременно с этим уменьшив влияние преимущества по оставшимся стенам ($\lambda_4 : 18 \rightarrow 8$) на общую оценку состояния игры.

3.2. Случайный алгоритм

`RandomAlgorithm` является простейшим из возможных программных соперников. При каждом вызове метода `getMove` алгоритм первым делом применяет эндшпильное правило: если можно немедленно выиграть самому или срочно заблокировать путь для соперника, он делает это независимо от уровня сложности. Если такое правило не сработало, поведение определяется уровнем сложности.

На уровне `EASY` алгоритм с равной вероятностью выбирает ход из полного множества допустимых ходов.

На уровне сложности `MEDIUM` ходы сортируются по убыванию значения оценки хода, и случайно выбирается один из первой списка ходов, ограниченного первой его половиной.

На уровне `HARD` случайно выбирается уже один из трёх лучших ходов с помощью применения той же оценки.

Общая схема работы алгоритма приведена в алгоритме 1.

Алгоритм 1: Случайный алгоритм

```
1  эндшпильный ход  $\leftarrow$  findEndgameMove( $s$ );
2  if эндшпильный ход найден then
3    |   return эндшпильный ход
4  end
5   $M \leftarrow$  getMoves( $s$ );
6  if уровень сложности = EASY then
7    |    $m^* \leftarrow$  случайный элемент  $M$ ;
8  else if уровень сложности = MEDIUM then
9    |   сортировать  $M$  по убыванию evaluateAfterMove;
10   |    $m^* \leftarrow$  случайный элемент из первой половины  $M$ ;
11 else
12   |   сортировать  $M$  по убыванию evaluateAfterMove;
13   |    $m^* \leftarrow$  случайный элемент из первых трёх элементов  $M$ ;
14 end
15 return  $m^*$ 
```

3.3. Минимакс

В MinimaxAlgorithm во время хода алгоритма выбирается ход с максимальной оценкой (максимизирующий узел), на ходах пользователя предполагается выбор хода с минимальной оценкой (минимизирующий узел).

Алгоритм итеративно увеличивает глубину анализа хода: последовательно запускает поиск с глубиной $d = 1, 2, \dots, d_{\max}$ до истечения лимита времени. На каждом этапе цикла в специальное поле сохраняется лучший выбранный ход, значение которого будет возвращено в качестве ответа. Благодаря этому, даже если лимит времени оказался исчерпан ранее, чем поиск на очередной глубине подошел к концу, всегда будет допустимый ответ, который можно будет использовать как ход алгоритма.

Максимальная глубина $d_{\max}$ и лимит времени T зависят от выбранных пользователем уровня сложности алгоритма и размера игрового поля:

Таблица 3.3

Параметры минимакса

Уровень	$d_{\max}$ (поле 9×9)	T , мс (поле 9×9)
EASY	3	700
MEDIUM	4	1800
HARD	5	3600

Чтобы алгоритм успевал обрабатывать большие значения глубины поиска за отведенное время, было введено ограничение на количество ходов для перебора на каждой итерации. После получения всего множества возможных вариантов хода из текущей позиции, эти ходы сортируются с помощью функции оценки, а

затем уже отсортированный список обрезается первыми 16 вхождениями в него. Уже обработанные алгоритмом позиции и их оценки хранятся в таблице *cache*. Общая схема работы алгоритма приведена в алгоритме 2.

Алгоритм 2: Минимакс с итеративным углублением

```

1  Функция minimax( $s, d, isMax$ ):
2      if в cache есть значение для ( $s, d, isMax$ ) then
3          return значение из cache;
4      end
5      if terminal( $s$ ) then
6          return терминальную оценку позиции;
7      end
8      if  $d = 0$  then
9          return evaluate( $s$ );
10     end
11      $p \leftarrow$  игрок, которому принадлежит ход в узле  $s$ ;
12      $M \leftarrow$  лучшие 16 ходов из getMoves( $s, p$ );
13     if  $isMax$  then
14          $best \leftarrow -\infty$ ;
15         foreach  $m \in M$  do
16              $best \leftarrow \max(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{false}))$ ;
17         end
18     else
19          $best \leftarrow +\infty$ ;
20         foreach  $m \in M$  do
21              $best \leftarrow \min(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{true}))$ ;
22         end
23     end
24     сохранить  $best$  в cache для ( $s, d, isMax$ );
25     return  $best$ ;

26  $m^* \leftarrow$  первый допустимый ход;
27 for  $d \leftarrow 1$  to  $d_{\max}$  do
28     if истёк лимит времени  $T$  then
29         break
30     end
31      $m^* \leftarrow \arg \max_{m \in M(s)} (\text{minimax}(\text{apply}(s, m), d - 1, \text{false}) +$ 
        movementPreference( $m, s$ ));
32 end
33 return  $m^*$ 

```

3.4. Альфа-бета отсечения

AlphaBetaAlgorithm является наиболее сложным из реализованных. Он представляет собой улучшенный Минимакс с несколькими независимыми механизмами ускорения: отсечениями ветвей дерева игры и сортировкой ходов [3].

Альфа-бета отсечение

Алгоритм использует два следующих параметра: α — это наилучшая оценка, которая гарантирована максимизирующему игроку, а β — это наилучшая оценка, гарантированная минимизирующему. Если в ходе перебора оказывается, что $\beta \leq \alpha$, оставшиеся ходы в данном узле дерева можно не рассматривать: они не изменят итогового выбора. Это позволяет уменьшить число рассматриваемых состояний и благодаря этому достичь большей глубины поиска хода.

Сортировка ходов

Эффективность отсечений ветвей критически зависит от порядка рассмотрения ходов: если лучший ход рассматривается первым, отсечения происходят как можно раньше, что оптимизирует затраченное время. Функция сортировки `scoreMove` оценивает каждый ход до его применения по нескольким критериям:

- $+10\,000 / +8\,000$, если ход является первым или вторым *killer-ходом* данного уровня глубины;
- вес из таблицы `goodMoves` — истории ходов, вызывавших отсечения в предыдущих узлах;
- значение функции оценки после применения хода.

Под *killer-ходом* понимается ход, который на проверке той же глубины поиска в дереве ранее уже привёл к отсечению $\beta \leq \alpha$. Такой ход считается перспективным не потому, что он обязательно является лучшим в текущей позиции, а потому что в похожих по глубине узлах он уже оказался достаточно сильным, чтобы сделать дальнейший перебор ветви бессмысленным. В реализации алгоритма для каждого уровня глубины хранится до двух таких ходов в историческом массиве `bestMoves`, при сортировке они получают высокий приоритет и рассматриваются раньше остальных.

Таблица `goodMoves` решает похожую задачу, но хранит не два последних хода для конкретной глубины, а накопленную статистику по строковому представлению ходов. Каждый раз, когда ход приводит к отсечению, его счётчик увеличивается. При следующих сортировках ход с наибольшим числом успешных отсечений получает небольшой дополнительный вес.

После сортировки список ходов обрезается до 36 кандидатов, по аналогии с алгоритмом Минимакс.

Поиск стабильной терминальной позиции

Обычный поиск с ограничением глубины имеет одну важную проблему: он может остановиться в момент, когда позиция ещё не является стабильной. Например, алгоритм дошёл до предельной глубины и видит, что путь до цели стал короче, но не успел проверить, что соперник следующим ходом тоже может выйти на финишную линию или поставить решающую стену. В такой ситуации простая статическая оценка может быть слишком оптимистичной и не может гарантировать победу алгоритма.

Для уменьшения этой ошибки в AlphaBetaAlgorithm используется поиск стабильной терминальной позиции. Его идея состоит в том, что на предельной глубине алгоритм не всегда сразу вызывает `evaluate()`, а сначала проверяет, не является ли позиция «острой». Таковой считается позиция, в которой хотя бы один из игроков находится не дальше двух ходов от победы.

Если позиция признана «острой», алгоритм выполняет небольшое дополнительное расширение дерева: рассматривает немедленные победные ходы и до восьми наиболее ценных установок стен, которые сильнее всего увеличивают путь соперника до цели. После этого для оставшихся «спокойных» позиций снова используется обычная функция оценки.

Параметры алгоритма в зависимости от уровня сложности для поля 9×9 :

Таблица 3.4

Параметры альфа-бета алгоритма

Уровень	$d_{\max}$	T , мс
EASY	4	1300
MEDIUM	6	3600
HARD	8	7600

Общая схема работы альфа-бета алгоритма приведена в алгоритме 3.

Алгоритм 3: Альфа-бета

```
1 Функция alphabeta( $s, d, \alpha, \beta, isMax$ ):
2   if terminal( $s$ ) then
3     return терминальная оценка
4   end
5   if  $d = 0$  then
6     if isNoisyPosition( $s$ ) then
7       return quiescence( $s, \alpha, \beta, isMax, 2$ )
8     else
9       return  $F(s)$ 
10    end
11  end
12   $M \leftarrow$  лучшие 36 ходов из getMoves( $s$ );
13  best  $\leftarrow -\infty$  если isMax, иначе  $+\infty$ ;
14  foreach  $m \in M$  do
15     $v \leftarrow$  alphabeta(apply( $s, m$ ),  $d - 1, \alpha, \beta, \neg isMax$ );
16    обновить best,  $\alpha$  или  $\beta$ ;
17    if  $\beta \leq \alpha$  then
18      обновить killer-ходы и goodMoves;
19      break ;                                // отсечение
20    end
21  end
22  return best
```

3.5. Монте-Карло

Алгоритм MonteCarloAlgorithm строит статистику по многократным симуляциям игры с текущей позиции фишки на игровом поле до конца партии [4]. В отличие от уже рассмотренных алгоритмов, он не перебирает дерево равномерно до фиксированной глубины, а постепенно концентрирует свои вычисления на наиболее перспективных ветвях. Структура алгоритма — дерево узлов, каждый из которых соответствует некоторому состоянию игры и хранит счётчики посещений узла во время проверки n_v и побед w_v , которые были достигнуты с посещением этого узла.

Основной цикл алгоритма состоит из четырёх фаз.

3.5.1. Выбор

Начиная с корня, алгоритм спускается по дереву, на каждом шаге выбирая дочерний узел с максимальным значением УСТ. Этот показатель нужен для баланса между двумя задачами: продолжать проверять ходы, которые уже часто

приводили к победе, а также исследовать менее изученные ходы.

$$\text{UCT}(v) = \frac{w_v}{n_v} + C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}} + h(v),$$

Первое слагаемое $\frac{w_v}{n_v}$ отражает уже накопленную статистику: чем чаще симуляции из узла v заканчивались победой, тем выше его оценка.

Второе слагаемое $C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}}$ отвечает за исследование. Если дочерний узел посещался редко, знаменатель n_v мал, поэтому значение этой части растёт и алгоритм получает стимул попробовать такой ход ещё раз. При увеличении числа посещений бонус постепенно уменьшается, и решение всё сильнее зависит от реальной статистики побед. Константа $C = 1,2$ задаёт силу исследования: при большем значении алгоритм активнее проверяет разные ветви, при меньшем быстрее концентрируется на уже найденных хороших ходах.

Третье слагаемое $h(v)$ является дополнительной эвристической поправкой, зависящей от функции оценки позиции:

$$h(v) = 0,15 \cdot \tanh(F(s_v)/500).$$

Она добавляет в формулу длины путей до цели, мобильность фишек игроков, оставшиеся лимиты стен и эндшпильные признаки. При этом коэффициент $0,15$ и функция $\tanh$ ограничивают влияние эвристики, чтобы она не подавляла статистику симуляций.

Если узел ни разу не посещался, ему присваивается приоритет $+\infty$. Благодаря этому каждый новый допустимый ход хотя бы один раз попадает в симуляцию.

3.5.2. Расширение

Когда все допустимые ходы из узла уже рассматривались в дочерних узлах, выбранный узел помечается как полностью расширенный. Иначе из списка ещё не рассмотренных ходов (предварительно отсортированных по `scoreMove`) берётся первый, создаётся новый дочерний узел и начинается его рассмотрение.

3.5.3. Симуляция

Из нового узла стартует случайная игра до терминального состояния или до исчерпания лимита шагов `rollOutDepth`. На каждом из ходов текущей симуляции применяется следующий механизм:

1. если есть победный ход, немедленно завершающий партию — используется он;
2. если соперник находится не далее 2 ходов от победы и у текущего игрока еще остались стены в запасе выбирается стена с максимальным `wallImpact`;

3. с вероятностью 0,22 выбирается случайный ход из полного списка;
4. иначе из случайной выборки размером 18 ходов выбирается один из четырёх лучших по scoreRolloutMove.

3.5.4. Обратное распространение

Результат сыгранной симуляции распространяется вверх: для каждого узла на пути от текущей позиции к корню дерева проставляются актуальные значения n_v и w_v .

3.5.5. Выбор лучшего хода

После истечения лимита времени или итераций алгоритма выбирается ход с наибольшим значением $n_c + w_c/n_c$ среди дочерних узлов корня, что отдаёт предпочтение хорошо исследованным ходам.

Параметры алгоритма для поля 9×9 :

Таблица 3.5

Параметры Монте-Карло

Уровень	Бюджет итераций	T , мс	Глубина симуляции
EASY	520	950	$9 \cdot 4 = 36$
MEDIUM	1700	2900	$9 \cdot 7 = 63$
HARD	3400	6200	$9 \cdot 10 = 90$

Алгоритм 4: Монте-Карло

```

1  $r \leftarrow$  корневой узел с состоянием  $s$ ;
2 while не истёк лимит  $T$  и итераций  $< N$  do
3    $v \leftarrow \text{select}(r)$ ;                                     // спуск по УСТ
4   if  $\neg \text{terminal}(v.s)$  then
5      $v \leftarrow \text{expand}(v)$ ;                                // новый дочерний узел
6   end
7    $\text{result} \leftarrow \text{simulate}(v, \text{rolloutDepth})$ ;
8    $\text{backpropagate}(v, \text{result})$ ;
9   if  $r.n > 300$  и  $\max_c(w_c/n_c) > 0.97$  then
10    break
11  end
12 end
13 return  $\arg \max_{c \in \text{children}(r)} (n_c + w_c/n_c)$ 
```

Условие досрочного выхода из узла дерева ($\text{bestWinRate} > 0.97$ при более чем 300 посещениях корня) позволяет экономить время в позициях, где один из ходов явно доминирует по статистике.

3.6. Сводные параметры сложности

Для удобства настройки экспериментов сравнения алгоритмов все уровни сложности сведены в таблицу 3.6. Значения приведены для стандартного поля 9×9 как базовые параметры приложения. Для полей 7×7 и 11×11 параметры масштабируются в коде: на малом поле AlphaBeta может увеличивать глубину поиска, а на большом поле MiniMax и AlphaBeta уменьшают глубину либо увеличивают временной лимит, чтобы партия оставалась интерактивной.

Таблица 3.6

Параметры уровней сложности алгоритмов для поля 9×9

Алгоритм	Уровень	Параметры
Random	EASY	Случайный выбор среди всех допустимых ходов
Random	MEDIUM	Случайный выбор из верхней половины ходов по функции оценки
Random	HARD	Случайный выбор из трёх лучших ходов по функции оценки
MiniMax	EASY	Глубина 3, лимит 700 мс, ширина перебора до 16 ходов
MiniMax	MEDIUM	Глубина 4, лимит 1800 мс, ширина перебора до 16 ходов
MiniMax	HARD	Глубина 5, лимит 3600 мс, ширина перебора до 16 ходов
AlphaBeta	EASY	Глубина 4, лимит 1300 мс, сортировка ходов, до 36 кандидатов
AlphaBeta	MEDIUM	Глубина 6, лимит 3600 мс, приоритет ходов, ранее приводивших к отсечениям
AlphaBeta	HARD	Глубина 8, лимит 7600 мс, поиск стабильности позиции в острых состояниях
MCTS	EASY	До 520 итераций, лимит 950 мс, глубина rollout $9 \cdot 4 = 36$
MCTS	MEDIUM	До 1700 итераций, лимит 2900 мс, глубина rollout $9 \cdot 7 = 63$
MCTS	HARD	До 3400 итераций, лимит 6200 мс, глубина rollout $9 \cdot 10 = 90$

4. Сравнение алгоритмов

4.1. Методика сравнения

Сравнение алгоритмов выполняется в отдельном режиме игрового приложения. Цель добавления режима — получить не единичный результат отдельной партии, а агрегированную оценку поведения стратегий на одинаковых условиях.

В эксперименте участвуют все применяемые в игровом приложении «Коридор» алгоритмы. Они сравниваются попарно, количество различных пар равно 6.

Для каждой пары играется заданное пользователем число партий. Для компенсации преимущества первого хода роли игроков чередуются: часть партий алгоритм проводит за первого игрока, часть — за второго. Размер поля и уровень сложности каждого алгоритма задаются перед запуском эксперимента.

Важно отметить, что в режиме сравнения алгоритмов обучение весов функции оценки не выполняется.

4.2. Собираемые метрики

Для анализа недостаточно знать только победителя. Два алгоритма могут иметь близкую долю побед, но существенно отличаться по времени работы, глубине поиска или количеству просмотренных состояний игрового поля. Поэтому приложение собирает несколько параметров статистики.

Метрики глубины, узлов и отсечений особенно важны для сравнения алгоритмов `MiniMaxAlgorithm` и `AlphaBetaAlgorithm`. Оба алгоритма основаны на дереве игры, но альфа-бета должен просматривать меньше узлов при большей достижимой глубине за счёт отсечений. Для `MonteCarloAlgorithm` ключевыми являются число итераций и время хода, поскольку качество решения определяется количеством проведённых симуляций.

4.3. План эксперимента

Итоговый эксперимент проводится на большом поле 11×11 , поскольку такой размер увеличивает пространство возможных ходов, делает партии длиннее и сильнее проявляет различия между стратегиями. Для всех алгоритмов устанавливается уровень сложности `HARD`. Для каждой пары алгоритмов проводится 50 партий. Так как сравниваются четыре алгоритма, всего рассматривается шесть пар, а суммарное число партий равно 300.

Таблица 4.1

Метрики сравнения алгоритмов

Метрика	Смысл
Количество партий	Общее число завершённых партий с участием алгоритма
Победы и поражения	Основная игровая результативность алгоритма
Доля побед	Доля побед алгоритма среди всех сыгранных им партий
Среднее число ходов	Косвенная характеристика стиля игры и сложности партий
Среднее время хода	Практическая вычислительная стоимость алгоритма
Средняя глубина	Насколько глубоко алгоритм успевает просмотреть дерево
Среднее число узлов	Объём реально просмотренного дерева поиска
Среднее число отсечений	Эффективность alpha-beta оптимизации

Каждый алгоритм участвует в трёх попарных сравнениях, поэтому итоговая статистика для одного алгоритма агрегируется по 150 партиям. Роли первого и второго игрока чередуются между партиями, поэтому при 50 партиях на пару каждый алгоритм начинает одинаковое число раз.

4.4. Форма представления результатов

Итоговая статистика представляется в двух формах. Первая форма — стандартная турнирная таблица.

Таблица 4.2

Турнирная таблица попарных результатов

Алгоритм	Случайный	MiniMax	MonteCarlo	AlphaBeta
Случайный	—	—	—	—
MiniMax	—	—	—	—
MonteCarlo	—	—	—	—
AlphaBeta	—	—	—	—

Вторая форма — агрегированная таблица по каждому алгоритму. Она удобнее для общего вывода, поскольку показывает не только победы, но и вычислительные затраты.

Таблица 4.3

Сводная статистика алгоритмов

Алгоритм	Партий	Победы	Поражения	Доля побед	Очки/партия	Ср. ходов	Ср. мс	Ср. глуб.	Ср. узлы	Ср. отсеч.
Случайный	150	–	–	–	–	–	–	–	–	–
MiniMax	150	–	–	–	–	–	–	–	–	–
MonteCarlo	150	–	–	–	–	–	–	–	–	–
AlphaBeta	150	–	–	–	–	–	–	–	–	–

4.5. Сравнение весов функции оценки

Для проверки полезности механизма обновления весов функции оценки проводится отдельный эксперимент. Один и тот же алгоритм запускается против своей копии, причем одна копия использует веса, которые были выданы функции оценки до начала их обновления, `EvaluationWeights.defaults()`, а другая — веса, полученные после self-play обучения и сохранённые в `evaluationweights.properties`. Такой эксперимент изолирует влияние весов: правила игры, алгоритм поиска, глубина и лимиты времени остаются одинаковыми.

Играется 50 партий на поле 11×11 с одинаковым уровнем сложности HARD для обеих вариаций алгоритма, роли первого и второго игрока чередуются.

Таблица 4.4

Проверка обновлённых весов функции оценки

Алгоритм	Партий	Победы начальных весов	Победы обновлённых весов
AlphaBeta	50	–	–

4.6. Анализ результатов

После выполнения эксперимента в данный раздел вносятся фактические результаты. Анализ строится по двум направлениям: игровая сила алгоритма и вычислительная стоимость принятия решения. Случайный алгоритм используется как базовая линия сравнения. MiniMax и AlphaBeta сопоставляются как два переборных алгоритма, а MonteCarlo анализируется отдельно, поскольку его качество определяется числом симуляций и устойчивостью UCT-выбора.

5. Игровое приложение

5.1. Техническая реализация

Игровое приложение «Коридор» написано на языке программирования Java версии 17 с использованием фреймворка Spring Boot версии 3.2.5, а также библиотеки Vaadin версии 24.4.2 [5], которая отвечает за веб-интерфейс. Для сокращения шаблонного кода используется библиотека Lombok. Система сборки кода — Maven.

Архитектура приложения разделена на несколько независимых слоёв. Общая схема взаимодействия слоёв представлена на рис. 4.

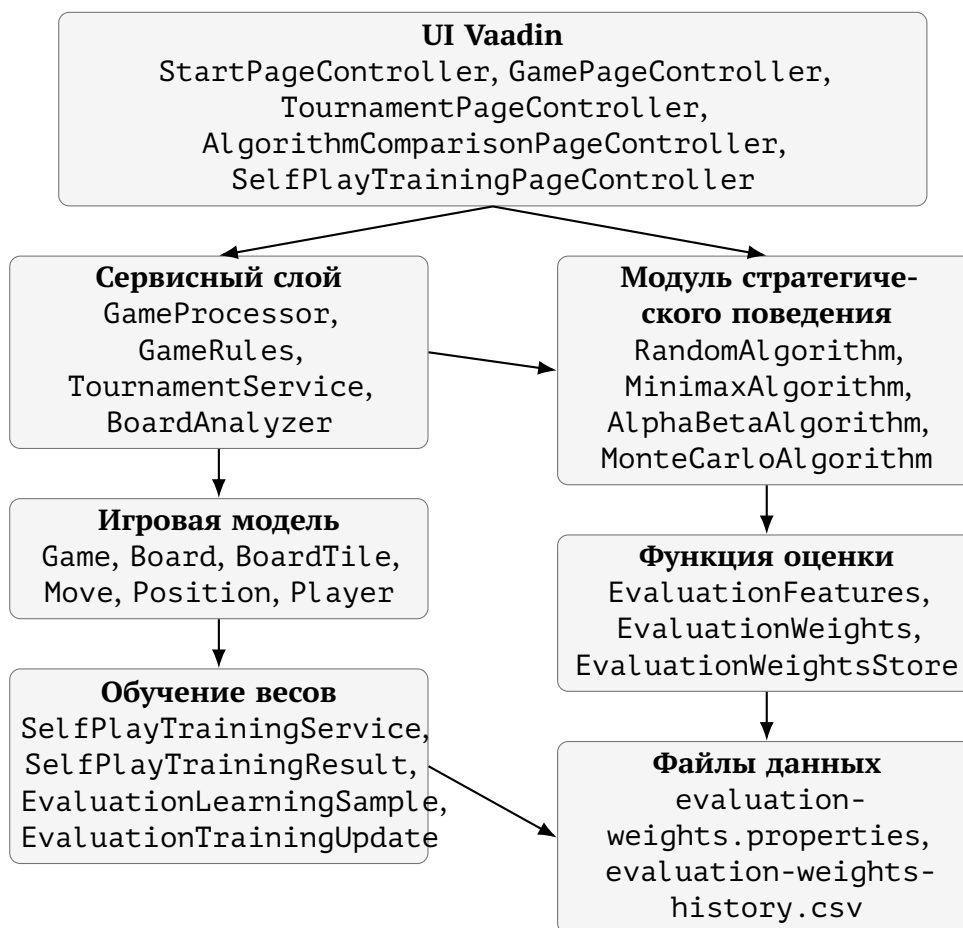


Рис. 4 — Общая архитектура приложения

Модель содержит классы, описывающие игровые объекты. Класс `Position` хранит координаты клетки (i, j) и предоставляет метод `move(rowDelta, colDelta)` для вычисления соседней позиции. Класс `BoardTile` описывает одну клетку поля четырьмя булевыми флагами доступности переходов. Класс `Board` хранит граф поля в виде `HashMap<Position, BoardTile>`, позиции фишек обоих игроков и реализует все операции с доской: размещение перегородок (`placeWall`), получе-

ние доступных ходов (`getAvailableMoves`) и полное копирование (`copy`). Класс `Move` описывает ход: его тип (`MOVE_PLAYER` или `PLACE_WALL`), идентификатор игрока и позиции начала и конца перемещения или стены. Класс `Game` хранит конфигурацию текущей партии: размер игрового поля (`GameSize`), объекты игроков, номер текущего хода, время начала игры и флаг ее завершения.

Игроки реализуют иерархию через абстрактный класс `Player` с методом `getMove`. Класс `HumanPlayer` возвращает `null`, поскольку ход игрока получается через его взаимодействие с интерфейсом игрового приложения; класс `ComputerPlayer` делегирует вызов конкретному объекту `Algorithm`. Фабрика `ComputerPlayerFabric` создаёт нужную реализацию алгоритма по типу и уровню сложности в соответствии с предпочтениями пользователя приложения.

Алгоритмы описаны в главе 2. Каждый из них реализует интерфейс `Algorithm`. Класс `AlgorithmReport` является неизменяемой записью с результатами последнего хода: выбранный ход, его оценка, достигнутая глубина поиска, количество рассмотренных узлов, количество отсечений ветвей дерева игры, время в миллисекундах, затраченное на анализ хода и его текстовое пояснение. Класс `EvaluationFeatures` хранит вектор признаков состояния; `EvaluationWeights` — веса; `EvaluationWeightsStore` управляет загрузкой и сохранением весов для историчности.

Сервисный слой включает несколько компонентов. `BoardFactory` создаёт начальное состояние доски заданного размера. Класс `GameRules` проверяет корректность установки стены и конвертирует координаты поля на пользовательском интерфейсе в игровые координаты, понятные алгоритмам. `GameProcessor` — аннотированный `@SessionScope` Spring-компонент, управляющий всем жизненным циклом партии: запуск (`startNewGame`), выполнение ходов (`makeMove`, `makeComputerMove`), сбор образцов для обучения весов и получение подсказок от всех четырёх алгоритмов в случае запроса таковых от пользователя приложения. `BoardAnalyzer` формирует текстовые пояснения к ходам ИИ. `TournamentService` позволяет запускать автоматические серии партий для турнира и сравнения алгоритмов. `SelfPlayTrainingService` отвечает за обучение весов параметров функции оценки: проводит партии между алгоритмами, собирает объекты `EvaluationLearningSample`, передаёт их в `EvaluationWeightsStore` и возвращает результат обучения в виде объекта `SelfPlayTrainingResult`.

Интерфейсный слой реализован через библиотечные компоненты `Vaadin`. `StartPageController` отображает стартовый экран с выбором режима и параметров. `GamePageController` реализует основной экран игры: отрисовку игровой доски в виде сетки, историю ходов (`GameHistoryPanel`), кнопки управления. `TournamentPageController`, `AlgorithmComparisonPageController` и `SelfPlayTrainingPageController` реализуют отображение режимов турнира, сравнения алгоритмов и обучения весов функции оценки.

5.2. Взаимодействие с приложением

При запуске игрового приложения «Коридор» пользователь попадает на стартовый экран, на котором настраиваются параметры предстоящей партии (рис. 5).

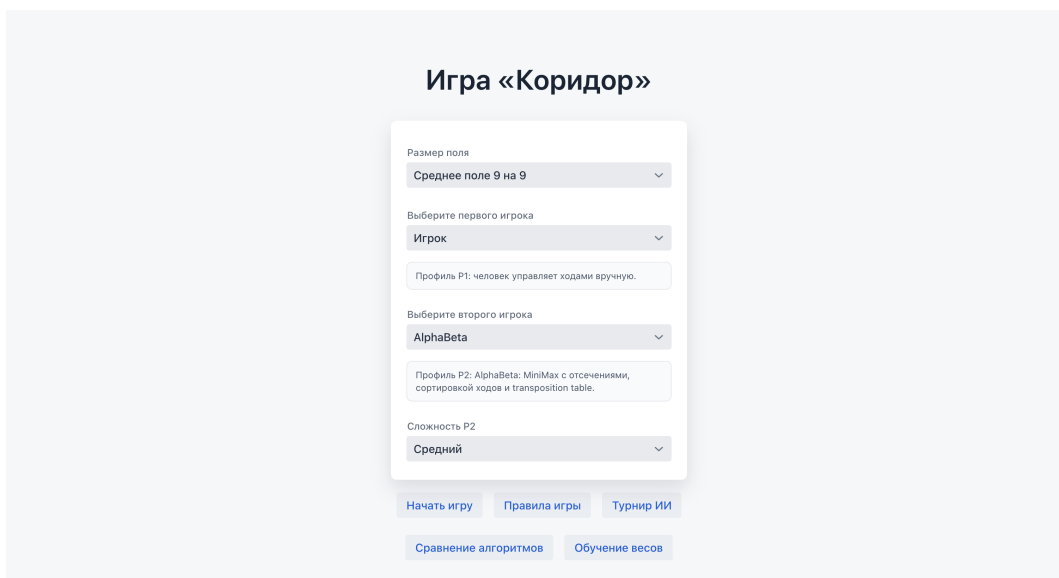


Рис. 5 — Стартовый экран выбора режима игры и алгоритмов

Доступны три варианта размера игрового поля: малое поле 7×7 с 8 стенами у каждого игрока, стандартное 9×9 с 10 стенами, большое 11×11 с 12 стенами. Размер поля влияет на все параметры алгоритмов: лимиты времени и максимальные глубины масштабируются пропорционально.

Первым игроком может управлять человек или один из четырёх алгоритмов программного соперника. Если выбран алгоритм, дополнительно настраивается его уровень сложности.

Второй игрок всегда управляется алгоритмом с настройкой его сложности. Карточки профилей отображают краткое описание выбранной стратегии.

На игровом экране пользователь видит несколько областей. Пример этого экрана приведён на рис. 6.

Игровое поле отрисовывается в виде сетки: клетки чередуются с промежутками, в которых могут быть размещены стены, блокирующие передвижения между ними. Каждая клетка имеет размер 42–50 пикселей в зависимости от размера поля. Фишки отображаются цветными кружками разного цвета. Допустимые для перемещения клетки подсвечиваются зелёным при наведении. При наведении на щель между клетками предварительно подсвечивается потенциальная стена.

Перемещение фишки игрока выполняется кликом на целевую клетку. Если ход недопустим, появляется уведомление.

Установка стены выполняется кликом на промежуток между клетками. `GameRules.extractWall` определяет по координатам интерфейса начальную и

конечную клетки перегородки. Затем `GameRules.canPlaceWall` копирует доску, устанавливает стену и проверяет достижимость целевых линий обоих игроков через обход в ширину.

В режиме «Игрок против ИИ» после хода пользователя автоматически запускается выбранный алгоритм. В режиме «ИИ против ИИ» каждый ход требует нажатия кнопки «Сделать ход ИИ», что позволяет наблюдать за партией пошагово, а не видеть лишь результат игры. Рядом с кнопкой отображается, чья очередь делать ход.

Кнопка «Подсказка хода» запускает все четыре алгоритма на текущей позиции и подсвечивает рекомендованные ими ходы разными цветами, каждый из которых соответствует одному из алгоритмов. Пример отображения подсказок приведён на рис. 6.

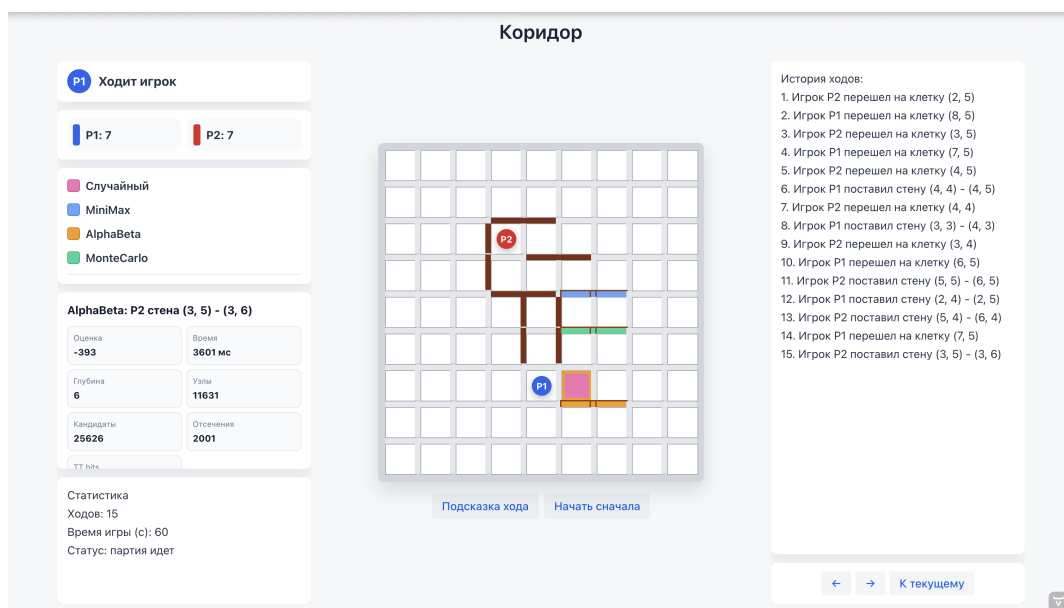


Рис. 6 — Отображение подсказок от алгоритмов на игровом поле

Панель информации отображает: чья очередь хода, количество оставшихся стен у каждого игрока, соответствие цветов и алгоритмов, а также отчёт о последнем ходе ИИ. Помимо прочего можно наблюдать за статистикой партии: числом ходов и затраченном времени.

С помощью кнопок «←», «→» и «К текущему» можно посмотреть историю ходов. Она также дублируется текстовым списком в правой панели.

Турнирный режим позволяет выбрать два алгоритма, их уровни сложности, размер поля и число партий. Результаты выводятся в режиме реального времени: по завершении каждой партии обновляется счётчик побед. По завершении серии выводится сводная таблица. Пример работы режима показан на рис. 7.

Режим сравнения алгоритмов автоматически проводит серию матчей между всеми попарными комбинациями четырёх алгоритмов и выводит сводную таблицу итогов с процентом побед, средним числом ходов, средним временем

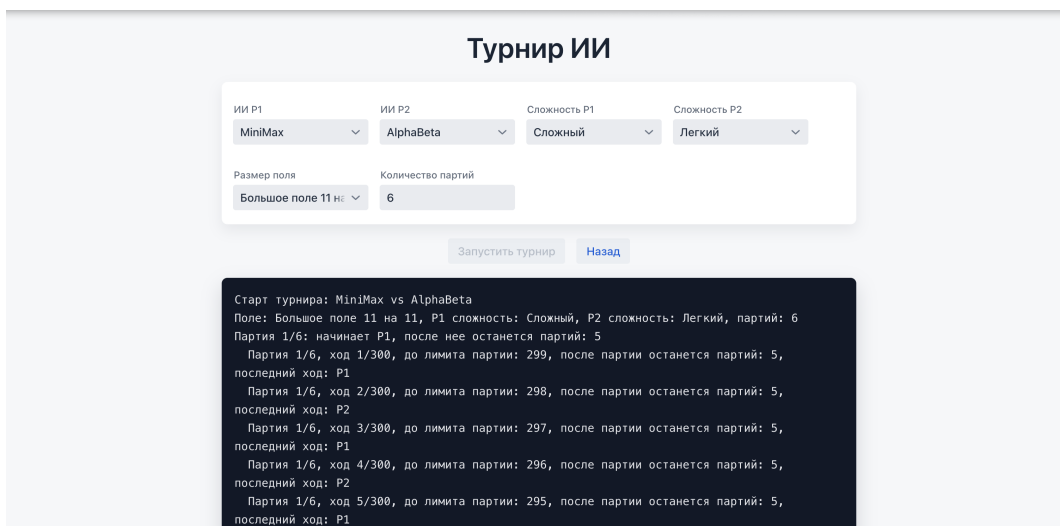


Рис. 7 — Экран турнирного режима ИИ

хода, средней глубиной поиска, числом узлов и отсечений. Интерфейс режима приведён на рис. 8.

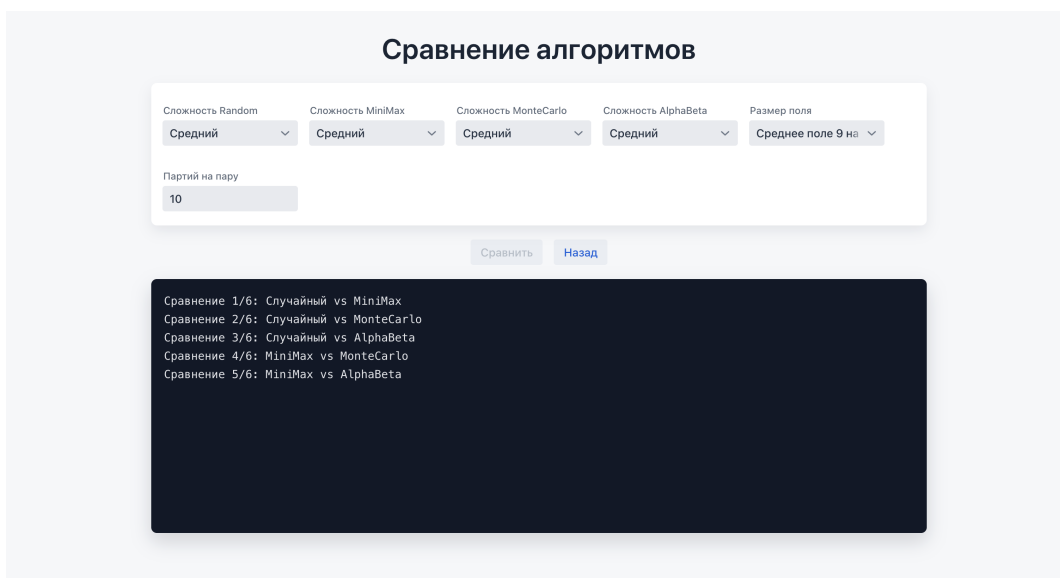


Рис. 8 — Экран сравнения алгоритмов

Режим обучения весов запускает серию партий между двумя выбранными алгоритмами. По завершении каждой партии сервис обучения обновляет веса функции оценки, сохраняет их в файл и выводит ход эксперимента. Пример обучения показан на рис. 9.

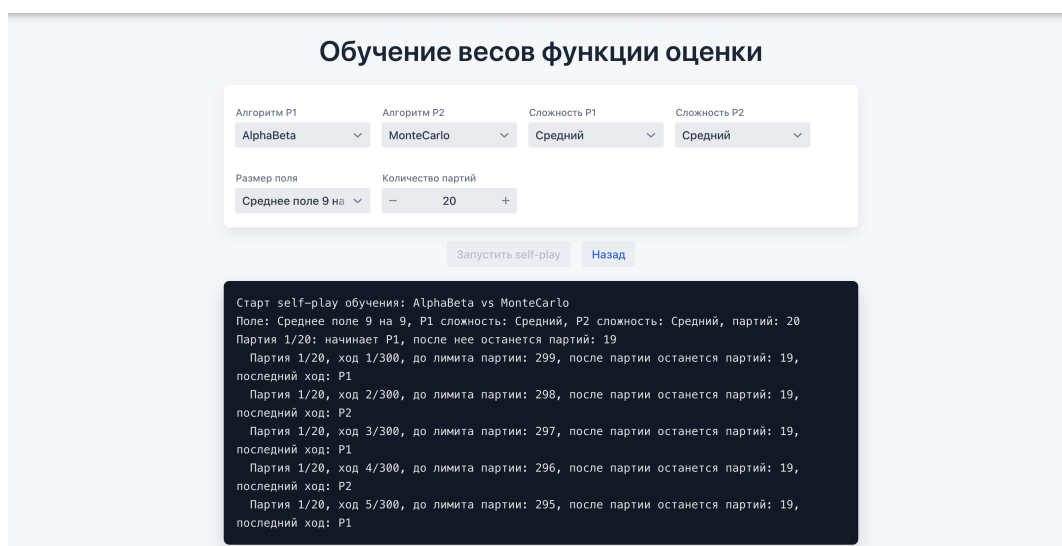


Рис. 9 — Экран self-play обучения весов функции оценки

Заключение

В ходе выполнения выпускной квалификационной работы была исследована настольная логическая игра «Коридор». Реализованы алгоритмы проверки корректности перемещения фишки с учётом всех правил прыжков через соперника и проверки корректности установки стены с помощью обхода графа в ширину. Реализован алгоритм поиска кратчайшего пути до целевой линии с восстановлением пути.

Разработана функция оценки позиции, учитывающая семь параметров. Реализован механизм коррекции весов функции оценки по результатам завершённых партий с сохранением в файл конфигурации между сессиями.

Реализованы четыре алгоритма программного соперника с тремя уровнями сложности каждый.

Разработано игровое приложение с режимами «Игрок против ИИ», «ИИ против ИИ», режимом подсказок одновременно от всех четырёх алгоритмов, просмотром истории партии, турнирным режимом и режимом сравнения алгоритмов.

Список литературы

- [1] BoardGameGeek. Games 100 [Электронный ресурс]. — Режим доступа: https://boardgamegeek.com/wiki/page/Games_100 (дата обращения: 02.04.2026).
- [2] Games, Hobby. Коридор. Правила игры [Электронный ресурс]. — Режим доступа: https://hobbygames.ru/download/rules/Quoridor_Rules.pdf (дата обращения: 03.04.2026).
- [3] Russell, S. Artificial Intelligence: A Modern Approach [Text] / S. Russell, P. Norvig. — Hoboken : Pearson, 2021.
- [4] Coulom, R. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search [Текст] / R. Coulom // Computers and Games. — 2006. — С. 72–83.
- [5] Vaadin. Vaadin Flow Documentation [Электронный ресурс]. — Режим доступа: <https://vaadin.com/docs> (дата обращения: 05.03.2026).

Реализация алгоритма MiniMax

Листинг А.1 — Класс MinimaxAlgorithm

```

1  public class MinimaxAlgorithm implements Algorithm {
2      private final Map<String, Integer> cache = new HashMap<>();
3      private AlgorithmReport lastReport;
4      private long nodesVisited;
5      private long consideredMoves;
6      private List<Position> recentPositions = List.of();
7
8      @Override
9      public ComputerAlgorithmType getType() {
10         return ComputerAlgorithmType.MINIMAX;
11     }
12
13     @Override
14     public AlgorithmReport getLastReport() {
15         return lastReport;
16     }
17
18     @Override
19     public void setRecentPositions(List<Position>
20         recentPositions) {
21         this.recentPositions = recentPositions == null ? List.
22             of() : List.copyOf(recentPositions);
23     }
24
25     @Override
26     public Move getMove(Board board,
27         ComputerPlayerHardnessLevel hardnessLevel, int playerId,
28         int wallsLeft1, int wallsLeft2) {
29         long startedAt = System.currentTimeMillis();
30         int size = (int) Math.sqrt(board.getTiles().size());
31         long timeLimit = getTimeLimit(hardnessLevel, size);
32         long endTime = System.currentTimeMillis() + timeLimit;
33         int maxDepth = getMaxDepth(hardnessLevel, size);

```

```

31     cache.clear();
32     nodesVisited = 0;
33     consideredMoves = 0;
34
35     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
36     Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
37     if (tacticalMove != null) {
38         int tacticalScore = scoreMoveAfterApply(board,
            tacticalMove, playerId, size, wallsLeft1,
            wallsLeft2);
39         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove, tacticalScore,
            0, nodesVisited, 1, 0, 0, System.
                currentTimeMillis() - startedAt, "MiniMax_примен
ил_эндшпильное_правило:_немедленная_победа_или_с
рочная_блокировка");
40         return tacticalMove;
41     }
42
43     Move best = null;
44     int bestScore = 0;
45     int reachedDepth = 0;
46     for (int depth = 1; depth <= maxDepth; depth++) {
47         if (System.currentTimeMillis() > endTime) {
48             break;
49         }
50         SearchResult result = search(board, depth, playerId
            , size, wallsLeft1, wallsLeft2, endTime);
51         if (result.move() != null) {
52             best = result.move();
53             bestScore = result.score();
54             reachedDepth = depth;
55         }
56     }
57     lastReport = new AlgorithmReport(getType().
        getDescription(), best, bestScore, reachedDepth,
        nodesVisited, consideredMoves, 0, 0, System.
            currentTimeMillis() - startedAt, "MiniMax_перебрал_д

```

```

        ерево_без_alpha-beta_отсечений;_лимит_времени:_"+
        timeLimit + "_мс");
58     return best;
59 }
60
61 private int getMaxDepth(ComputerPlayerHardnessLevel level,
        int size) {
62     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
63     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
64     return switch (level) {
65         case EASY -> 3;
66         case MEDIUM -> smallBoard ? 5 : 4;
67         case HARD -> largeBoard ? 4 : 5;
68     };
69 }
70
71 @Override
72 public long getTimeLimit(ComputerPlayerHardnessLevel level,
        int size) {
73     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
74     return switch (level) {
75         case EASY -> largeBoard ? 900L : 700L;
76         case MEDIUM -> largeBoard ? 2200L : 1800L;
77         case HARD -> largeBoard ? 4200L : 3600L;
78     };
79 }
80
81 private SearchResult search(Board board, int depth, int
        playerId, int size, int wallsLeft1, int wallsLeft2, long
        endTime) {
82     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
83     List<Move> moves = orderedLimitedMoves(board, getMoves(
        board, playerId, currentWalls), playerId, size,
        wallsLeft1, wallsLeft2);
84     consideredMoves += moves.size();
85

```

```

86         Move bestMove = null;
87         int bestValue = Integer.MIN_VALUE;
88         for (Move move : moves) {
89             if (System.currentTimeMillis() > endTime) {
90                 return new SearchResult(bestMove, bestValue);
91             }
92
93             Board copy = board.copy();
94             applyMove(copy, move);
95
96             int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
                wallsLeft2;
97             if (move.getMoveType() == MoveType.PLACE_WALL) {
98                 if (playerId == 1) {
99                     --newWallsLeft1;
100                 } else {
101                     --newWallsLeft2;
102                 }
103             }
104
105             int value = minimax(copy, depth - 1, false,
106                 playerId, size, newWallsLeft1,
107                 newWallsLeft2, endTime);
108             value += movementPreference(move, board, playerId,
109                 size, recentPositions);
110             if (value > bestValue) {
111                 bestValue = value;
112                 bestMove = move;
113             }
114         }
115         return new SearchResult(bestMove, bestValue);
116     }
117
118     private int minimax(Board board, int depth, boolean
119         maximizing, int playerId, int size, int wallsLeft1, int
120         wallsLeft2, long endTime) {
121         nodesVisited++;
122         if (System.currentTimeMillis() > endTime) {
123             return evaluate(board, playerId, size, wallsLeft1,
124                 wallsLeft2);
125         }
126     }

```

```

120     }
121     String key = boardKey(board) + "/" + depth + "/" +
        maximizing + "/" + wallsLeft1 + "/" + wallsLeft2;
122     if (cache.containsKey(key)) {
123         return cache.get(key);
124     }
125
126     Integer terminal = terminalScore(board, playerId, size,
        depth);
127     if (terminal != null) {
128         cache.put(key, terminal);
129         return terminal;
130     }
131
132     if (depth == 0) {
133         int val = evaluate(board, playerId, size,
            wallsLeft1, wallsLeft2);
134         cache.put(key, val);
135         return val;
136     }
137
138     int current = maximizing ? playerId : (3 - playerId);
139     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
140     List<Move> moves = orderedLimitedMoves(board, getMoves(
        board, current, walls), playerId, size, wallsLeft1,
        wallsLeft2);
141     consideredMoves += moves.size();
142
143     int best = maximizing ? Integer.MIN_VALUE : Integer.
        MAX_VALUE;
144     for (Move move : moves) {
145         Board copy = board.copy();
146         applyMove(copy, move);
147
148         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
149         if (move.getMoveType() == MoveType.PLACE_WALL) {
150             if (current == 1) {
151                 --newWallsLeft1;
152             } else {

```

```

153         --newWallsLeft2;
154     }
155 }
156
157     int val = minimax(copy, depth - 1, !maximizing,
158         playerId, size, newWallsLeft1,
159         newWallsLeft2, endTime);
160
161     if (maximizing) {
162         best = Math.max(best, val);
163     } else {
164         best = Math.min(best, val);
165     }
166 }
167 cache.put(key, best);
168 return best;
169 }
170
171 private List<Move> orderedLimitedMoves(Board board, List<
172     Move> moves, int playerId, int size, int wallsLeft1, int
173     wallsLeft2) {
174     List<Move> ordered = new ArrayList<>(moves);
175     ordered.sort((a, b) -> Integer.compare(
176         scoreMoveForOrdering(board, b, playerId, size,
177         wallsLeft1, wallsLeft2),
178         scoreMoveForOrdering(board, a, playerId, size,
179         wallsLeft1, wallsLeft2)
180     ));
181     if (ordered.size() <= 16) {
182         return ordered;
183     }
184     return ordered.subList(0, 16);
185 }
186
187 private int scoreMoveForOrdering(Board board, Move move,
188     int playerId, int size, int wallsLeft1, int wallsLeft2)
189 {
190     return scoreMoveAfterApply(board, move, playerId, size,
191         wallsLeft1, wallsLeft2) + movementPreference(move,
192         board, playerId, size, recentPositions);
193 }

```

```

184
185     private int scoreMoveAfterApply(Board board, Move move, int
186         playerId, int size, int wallsLeft1, int wallsLeft2) {
187         Board copy = board.copy();
188         applyMove(copy, move);
189         int newWallsLeft1 = wallsLeft1;
190         int newWallsLeft2 = wallsLeft2;
191         if (move.getMoveType() == MoveType.PLACE_WALL) {
192             if (move.getPlayerId() == 1) {
193                 --newWallsLeft1;
194             } else {
195                 --newWallsLeft2;
196             }
197         }
198         return evaluate(copy, playerId, size, newWallsLeft1,
199             newWallsLeft2);
200     }
201 }

```

Реализация алгоритма AlphaBeta

Листинг Б.1 — Класс AlphaBetaAlgorithm

```

1 public class AlphaBetaAlgorithm implements Algorithm {
2     private final Map<String, TranspositionEntry>
        transpositionTable = new HashMap<>();
3     private final Map<String, Integer> goodMoves = new HashMap
        <>();
4     private final Move[][] bestMoves = new Move[50][2];
5     private AlgorithmReport lastReport;
6     private long nodesVisited;
7     private long consideredMoves;
8     private long cutoffs;
9     private long tableHits;
10    private List<Position> recentPositions = List.of();
11    private EvaluationWeights evaluationWeights;
12
13    @Override
14    public ComputerAlgorithmType getType() {
15        return ComputerAlgorithmType.ALPHABETA;
16    }
17
18    @Override
19    public AlgorithmReport getLastReport() {
20        return lastReport;
21    }
22
23    @Override
24    public void setRecentPositions(List<Position>
        recentPositions) {
25        this.recentPositions = recentPositions == null ? List.
            of() : List.copyOf(recentPositions);
26    }
27
28    @Override
29    public EvaluationWeights getEvaluationWeights() {
30        return evaluationWeights == null ? Algorithm.super.

```

```

        getEvaluationWeights() : evaluationWeights;
31     }
32
33     @Override
34     public void setEvaluationWeights(EvaluationWeights
        evaluationWeights) {
35         this.evaluationWeights = evaluationWeights;
36     }
37
38     @Override
39     public Move getMove(Board board,
        ComputerPlayerHardnessLevel level, int playerId, int
        wallsLeft1, int wallsLeft2) {
40         long startedAt = System.currentTimeMillis();
41         int size = (int) Math.sqrt(board.getTiles().size());
42         int maxDepth = getMaxDepth(level, size);
43         long timeLimit = getTimeLimit(level, size);
44         long endTime = System.currentTimeMillis() + timeLimit;
45
46         transpositionTable.clear();
47         nodesVisited = 0;
48         consideredMoves = 0;
49         cutoffs = 0;
50         tableHits = 0;
51
52         int currentWalls = playerId == 1 ? wallsLeft1 :
            wallsLeft2;
53         Move tacticalMove = findEndgameMove(board, playerId,
            currentWalls);
54         if (tacticalMove != null) {
55             int tacticalScore = scoreMoveAfterApply(board,
                tacticalMove, playerId, size, wallsLeft1,
                wallsLeft2);
56             lastReport = new AlgorithmReport(getType().
                getDescription(), tacticalMove, tacticalScore,
                0, nodesVisited, 1, cutoffs, tableHits, System.
                currentTimeMillis() - startedAt, "AlphaBeta_прим
                енил_эндшпильное_правило:_немедленная_победа_или
                _срочная_блокировка");
57             return tacticalMove;

```

```

58     }
59
60     Move bestMove = null;
61     int bestScore = 0;
62     int reachedDepth = 0;
63     for (int depth = 1; depth <= maxDepth; ++depth) {
64         if (System.currentTimeMillis() > endTime) {
65             break;
66         }
67         SearchResult result = search(board, depth, playerId
68             , size, wallsLeft1, wallsLeft2, endTime);
69         if (result.move() != null) {
70             bestMove = result.move();
71             bestScore = result.score();
72             reachedDepth = depth;
73         }
74     }
75     lastReport = new AlgorithmReport(getType().
76         getDescription(), bestMove, bestScore, reachedDepth,
77         nodesVisited, consideredMoves, cutoffs, tableHits,
78         System.currentTimeMillis() - startedAt, "AlphaBeta_и
79         использует_сортировку_ходов,_beta_<=_alpha_отсечения_
80         и_transposition_table_c_depth-bound_flags;_попаданий
81         _в_таблицу:_" + tableHits + ";_лимит_времени:_" +
82         timeLimit + "_мс");
83     return bestMove;
84 }
85
86 private int getMaxDepth(ComputerPlayerHardnessLevel level,
87     int size) {
88     boolean smallBoard = size <= GameSize.SMALL.
89         getAmountOfTilesPerSide();
90     boolean largeBoard = size > GameSize.NORMAL.
91         getAmountOfTilesPerSide();
92     return switch (level) {
93         case EASY -> smallBoard ? 5 : 4;
94         case MEDIUM -> smallBoard ? 7 : 6;
95         case HARD -> largeBoard ? 7 : 8;
96     };
97 }

```

```

87
88     @Override
89     public long getTimeLimit(ComputerPlayerHardnessLevel level,
90         int size) {
91         boolean largeBoard = size > GameSize.NORMAL.
92             getAmountOfTilesPerSide();
93         boolean smallBoard = size <= GameSize.SMALL.
94             getAmountOfTilesPerSide();
95         return switch (level) {
96             case EASY -> smallBoard ? 1000L : 1300L;
97             case MEDIUM -> largeBoard ? 4800L : 3600L;
98             case HARD -> largeBoard ? 9500L : 7600L;
99         };
100     }
101
102     private SearchResult search(Board board, int depth, int
103         playerId, int size, int wallsLeft1, int wallsLeft2, long
104         endTime) {
105         int currentWalls = playerId == 1 ? wallsLeft1 :
106             wallsLeft2;
107         List<Move> moves = new ArrayList<>(getMoves(board,
108             playerId, currentWalls).stream()
109                 .filter(move -> isRootMoveLegal(board, move,
110                     playerId, currentWalls)).toList());
111         orderMoves(moves, board, playerId, size, wallsLeft1,
112             wallsLeft2, 0, null);
113         if (moves.size() > 36) {
114             moves = moves.subList(0, 36);
115         }
116         consideredMoves += moves.size();
117
118         Move bestMove = null;
119         int best = Integer.MIN_VALUE;
120         for (Move move : moves) {
121             if (System.currentTimeMillis() > endTime) {
122                 break;
123             }
124             Board copy = board.copy();
125             applyMove(copy, move);

```

```

118         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
119         if (move.getMoveType() == MoveType.PLACE_WALL) {
120             if (playerId == 1) {
121                 --newWallsLeft1;
122             } else {
123                 --newWallsLeft2;
124             }
125         }
126         int val = alphaBeta(copy, depth - 1, Integer.
            MIN_VALUE, Integer.MAX_VALUE, false, playerId,
            size, newWallsLeft1, newWallsLeft2, endTime, 1);
127         val += rootMovePreference(move, board, playerId,
            size);
128         if (val > best) {
129             best = val;
130             bestMove = move;
131         }
132     }
133     return new SearchResult(bestMove, best);
134 }
135
136 private int alphaBeta(Board board, int depth, int alpha,
    int beta, boolean max, int playerId, int size, int
    wallsLeft1, int wallsLeft2, long endTime, int ply) {
137     nodesVisited++;
138     if (System.currentTimeMillis() > endTime) {
139         return evaluate(board, playerId, size, wallsLeft1,
            wallsLeft2);
140     }
141     String key = boardKey(board) + "/" + max + "/" +
        wallsLeft1 + "/" + wallsLeft2;
142     int originalAlpha = alpha;
143     int originalBeta = beta;
144
145     TranspositionEntry cached = transpositionTable.get(key)
        ;
146     if (cached != null && cached.depth >= depth) {
147         if (cached.bound == Bound.EXACT) {
148             tableHits++;

```

```

149         return cached.score;
150     }
151     if (cached.bound == Bound.LOWER) {
152         alpha = Math.max(alpha, cached.score);
153     } else if (cached.bound == Bound.UPPER) {
154         beta = Math.min(beta, cached.score);
155     }
156     if (alpha >= beta) {
157         tableHits++;
158         return cached.score;
159     }
160 }
161
162 Integer terminal = terminalScore(board, playerId, size,
    depth);
163 if (terminal != null) {
164     transpositionTable.put(key, new TranspositionEntry(
        depth, terminal, Bound.EXACT, null));
165     return terminal;
166 }
167
168 if (depth == 0) {
169     if (isNoisyPosition(board, playerId, size)) {
170         int calmScore = quiescence(board, alpha, beta,
            max, playerId, size, wallsLeft1, wallsLeft2,
            endTime, 2);
171         transpositionTable.put(key, new
            TranspositionEntry(depth, calmScore, Bound.
                EXACT, null));
172         return calmScore;
173     }
174     int eval = evaluate(board, playerId, size,
        wallsLeft1, wallsLeft2);
175     transpositionTable.put(key, new TranspositionEntry(
        depth, eval, Bound.EXACT, null));
176     return eval;
177 }
178
179 int current = max ? playerId : 3 - playerId;
180 int walls = current == 1 ? wallsLeft1 : wallsLeft2;

```

```

181
182     List<Move> moves = getMoves(board, current, walls);
183     Move tableBestMove = cached == null ? null : cached.
        bestMove;
184     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, ply, tableBestMove);
185     if (moves.size() > 36) {
186         moves = moves.subList(0, 36);
187     }
188     consideredMoves += moves.size();
189
190     int best = max ? Integer.MIN_VALUE : Integer.MAX_VALUE;
191     Move bestMove = null;
192
193     for (Move move : moves) {
194         Board copy = board.copy();
195         applyMove(copy, move);
196
197         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
198         if (move.getMoveType() == MoveType.PLACE_WALL) {
199             if (current == 1) {
200                 --newWallsLeft1;
201             } else {
202                 --newWallsLeft2;
203             }
204         }
205
206         int val = alphaBeta(copy, depth - 1, alpha, beta, !
            max, playerId, size, newWallsLeft1,
            newWallsLeft2, endTime, ply + 1);
207
208         if (max) {
209             if (val > best) {
210                 best = val;
211                 bestMove = move;
212             }
213             alpha = Math.max(alpha, val);
214         } else {
215             if (val < best) {

```

```

216             best = val;
217             bestMove = move;
218         }
219         beta = Math.min(beta, val);
220     }
221
222     if (beta <= alpha) {
223         cutoffs++;
224         updateBestMoves(move, ply);
225         updateGoodMoves(move);
226         break;
227     }
228 }
229
230 Bound bound = Bound.EXACT;
231 if (best <= originalAlpha) {
232     bound = Bound.UPPER;
233 } else if (best >= originalBeta) {
234     bound = Bound.LOWER;
235 }
236 transpositionTable.put(key, new TranspositionEntry(
237     depth, best, bound, bestMove));
238 return best;
239 }
240
241 private int quiescence(Board board, int alpha, int beta,
242     boolean max, int playerId, int size, int wallsLeft1, int
243     wallsLeft2, long endTime, int extensionDepth) {
244     nodesVisited++;
245     int standPat = evaluate(board, playerId, size,
246         wallsLeft1, wallsLeft2);
247     if (extensionDepth == 0 || System.currentTimeMillis() >
248         endTime) {
249         return standPat;
250     }
251
252     if (max) {
253         if (standPat >= beta) {
254             return beta;
255         }
256     }

```

```

251         alpha = Math.max(alpha, standPat);
252     } else {
253         if (standPat <= alpha) {
254             return alpha;
255         }
256         beta = Math.min(beta, standPat);
257     }
258
259     int current = max ? playerId : 3 - playerId;
260     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
261     List<Move> moves = getQuiescenceMoves(board, current,
        walls, playerId, size);
262     if (moves.isEmpty()) {
263         return standPat;
264     }
265     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, 0, null);
266
267     for (Move move : moves) {
268         if (System.currentTimeMillis() > endTime) {
269             break;
270         }
271         Board copy = board.copy();
272         applyMove(copy, move);
273
274         int newWallsLeft1 = wallsLeft1;
275         int newWallsLeft2 = wallsLeft2;
276         if (move.getMoveType() == MoveType.PLACE_WALL) {
277             if (current == 1) {
278                 --newWallsLeft1;
279             } else {
280                 --newWallsLeft2;
281             }
282         }
283
284         int value = quiescence(copy, alpha, beta, !max,
            playerId, size, newWallsLeft1, newWallsLeft2,
            endTime, extensionDepth - 1);
285         if (max) {
286             if (value >= beta) {

```

```

287             cutoffs++;
288             return beta;
289         }
290         alpha = Math.max(alpha, value);
291     } else {
292         if (value <= alpha) {
293             cutoffs++;
294             return alpha;
295         }
296         beta = Math.min(beta, value);
297     }
298 }
299 return max ? alpha : beta;
300 }
301
302 private void orderMoves(List<Move> moves, Board board, int
    playerId, int size, int wallsLeft1, int wallsLeft2, int
    ply, Move tableBestMove) {
303     moves.sort((a, b) -> Integer.compare(scoreMove(b, board
        , playerId, size, wallsLeft1, wallsLeft2, ply,
        tableBestMove), scoreMove(a, board, playerId, size,
        wallsLeft1, wallsLeft2, ply, tableBestMove)));
304 }
305
306 private int scoreMove(Move move, Board board, int playerId,
    int size, int wallsLeft1, int wallsLeft2, int ply, Move
    tableBestMove) {
307     int score = 0;
308
309     if (move.equals(tableBestMove)) {
310         score += 20000;
311     }
312
313     if (ply < bestMoves.length) {
314         if (bestMoves[ply][0] != null && move.equals(
            bestMoves[ply][0])) {
315             score += 10000;
316         }
317         if (bestMoves[ply][1] != null && move.equals(
            bestMoves[ply][1])) {

```

```

318         score += 8000;
319     }
320 }
321
322     score += goodMoves.getOrDefault(move.toString(), 0) *
323         2;
324     score += rootMovePreference(move, board, playerId, size
325         );
326     score += scoreMoveAfterApply(board, move, playerId,
327         size, wallsLeft1, wallsLeft2);
328
329     return score;
330 }
331
332 private int scoreMoveAfterApply(Board board, Move move, int
333     playerId, int size, int wallsLeft1, int wallsLeft2) {
334     Board copy = board.copy();
335     applyMove(copy, move);
336     int newWallsLeft1 = wallsLeft1;
337     int newWallsLeft2 = wallsLeft2;
338     if (move.getMoveType() == MoveType.PLACE_WALL) {
339         if (move.getPlayerId() == 1) {
340             --newWallsLeft1;
341         } else {
342             --newWallsLeft2;
343         }
344     }
345     return evaluate(copy, playerId, size, newWallsLeft1,
346         newWallsLeft2);
347 }
348
349 private int rootMovePreference(Move move, Board board, int
350     playerId, int size) {
351     return movementPreference(move, board, playerId, size,
352         recentPositions);
353 }
354
355 private boolean isRootMoveLegal(Board board, Move move, int
356     playerId, int wallsLeft) {
357     if (move.getPlayerId() != playerId) {

```

```

350         return false;
351     }
352     if (move.getMoveType() == MoveType.MOVE_PLAYER) {
353         return board.getAvailableMoves(getCurrentPosition(
            board, playerId)).contains(move.getEndPosition()
            );
354     }
355     if (wallsLeft <= 0) {
356         return false;
357     }
358     return isWallPlaceable(board, move.getStartPosition(),
        move.getEndPosition()) && isValidWall(board, move.
        getStartPosition(), move.getEndPosition());
359 }
360
361 private void updateBestMoves(Move move, int ply) {
362     if (ply >= bestMoves.length || move.equals(bestMoves[
        ply][0])) {
363         return;
364     }
365     bestMoves[ply][1] = bestMoves[ply][0];
366     bestMoves[ply][0] = move;
367 }
368
369 private void updateGoodMoves(Move move) {
370     goodMoves.merge(move.toString(), 1, Integer::sum);
371 }
372
373 private record SearchResult(Move move, int score) {}
374
375 private enum Bound {
376     EXACT,
377     LOWER,
378     UPPER
379 }
380
381 private record TranspositionEntry(int depth, int score,
    Bound bound, Move bestMove) {}
382 }

```

Реализация алгоритма Monte Carlo

Листинг В.1 — Класс MonteCarloAlgorithm

```

1 public class MonteCarloAlgorithm implements Algorithm {
2     private static final double C = 1.2;
3     private static final Random random = new Random();
4     private AlgorithmReport lastReport;
5     private List<Position> recentPositions = List.of();
6     private int rootPlayerId;
7
8     @Override
9     public ComputerAlgorithmType getType() {
10         return ComputerAlgorithmType.MONTECARLO;
11     }
12
13     @Override
14     public AlgorithmReport getLastReport() {
15         return lastReport;
16     }
17
18     @Override
19     public void setRecentPositions(List<Position>
20         recentPositions) {
21         this.recentPositions = recentPositions == null ? List.
22             of() : List.copyOf(recentPositions);
23     }
24
25     @Override
26     public Move getMove(Board board,
27         ComputerPlayerHardnessLevel hardnessLevel, int playerId,
28         int wallsLeft1, int wallsLeft2) {
29         long startedAt = System.currentTimeMillis();
30         int size = (int) Math.sqrt(board.getTiles().size());
31         long timeLimit = getTimeLimit(hardnessLevel, size);
32         long endTime = System.currentTimeMillis() + timeLimit;
33         int rolloutDepth = getRolloutDepth(hardnessLevel, size)
34             ;

```

```

30     long iterationBudget = getIterationBudget(hardnessLevel
        , size);
31
32     long iterations = 0;
33     rootPlayerId = playerId;
34     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
35     Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
36     if (tacticalMove != null) {
37         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove, scoreMove(board,
            tacticalMove, playerId, wallsLeft1, wallsLeft2)
            , 0, 0, 1, 0, 0, System.currentTimeMillis() -
            startedAt, "MCTS_применил_эндшпильное_правило_до
            _запуска_симуляций");
38         return tacticalMove;
39     }
40
41     Node root = new Node(board.copy(), null, null, playerId
        , wallsLeft1, wallsLeft2);
42     while (System.currentTimeMillis() < endTime &&
        iterations < iterationBudget) {
43         Node node = select(root);
44         Node expanded = expand(node);
45         int winner = simulate(expanded, rolloutDepth,
            playerId, size);
46         backpropagate(expanded, winner, playerId);
47         iterations++;
48         if (root.visits > 300 && bestWinRate(root) > 0.97)
            {
49             break;
50         }
51     }
52
53     Node best = root.children.stream()
54         .max(Comparator.comparingDouble(this::
            robustChildScore))
55         .orElseThrow();
56     lastReport = new AlgorithmReport(getType().

```

```

        getDescription(), best.move, evaluate(best.board,
        playerId, size, best.walls1, best.walls2),
        rolloutDepth, iterations, root.children.size(), 0,
        0, System.currentTimeMillis() - startedAt, "MCTS:_вы
        бор_по_UCT,_rollout_с_epsilon-greedy_эвристикой,_нем
        едленными_победами_и_тактическими_стенами;_лимит_вре
        мени:_" + timeLimit + "_мс,_бюджет_итераций:_" +
        iterationBudget);
57     return best.move;
58 }
59
60 private int getRolloutDepth(ComputerPlayerHardnessLevel
    level, int size) {
61     return switch (level) {
62         case EASY -> size * 4;
63         case MEDIUM -> size * 7;
64         case HARD -> size * 10;
65     };
66 }
67
68 private long getIterationBudget(ComputerPlayerHardnessLevel
    level, int size) {
69     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
70     return switch (level) {
71         case EASY -> largeBoard ? 380L : 520L;
72         case MEDIUM -> largeBoard ? 1300L : 1700L;
73         case HARD -> largeBoard ? 2600L : 3400L;
74     };
75 }
76
77 @Override
78 public long getTimeLimit(ComputerPlayerHardnessLevel level,
    int size) {
79     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
80     return switch (level) {
81         case EASY -> largeBoard ? 1200L : 950L;
82         case MEDIUM -> largeBoard ? 3800L : 2900L;
83         case HARD -> largeBoard ? 7600L : 6200L;

```

```

84         };
85     }
86
87     private Node select(Node node) {
88         while (!node.children.isEmpty() && node.untriedMoves.
            isEmpty()) {
89             node = node.children.stream().max(Comparator.
                comparing(this::uct)).orElseThrow();
90         }
91         return node;
92     }
93
94     private Node expand(Node node) {
95         if (node.untriedMoves.isEmpty() && !node.children.
            isEmpty()) {
96             return node;
97         }
98
99         if (node.untriedMoves.isEmpty()) {
100             node.untriedMoves.addAll(getMoves(node.board, node.
                player, node.getWalls()));
101             node.untriedMoves.sort((a, b) -> Integer.compare(
102                 scoreMove(node.board, b, node.player, node.
                    walls1, node.walls2),
103                 scoreMove(node.board, a, node.player, node.
                    walls1, node.walls2)
104             ));
105         }
106
107         if (node.untriedMoves.isEmpty()) {
108             return node;
109         }
110
111         Move move = node.untriedMoves.remove(0);
112         Board copy = node.board.copy();
113         applyMove(copy, move);
114
115         int wallsLeft1 = node.walls1;
116         int wallsLeft2 = node.walls2;
117

```

```

118         if (move.getMoveType() == MoveType.PLACE_WALL) {
119             if (node.player == 1) {
120                 --wallsLeft1;
121             } else {
122                 --wallsLeft2;
123             }
124         }
125         Node child = new Node(copy, node, move, 3 - node.player
126             , wallsLeft1, wallsLeft2);
127         node.children.add(child);
128         return child;
129     }
130     private int simulate(Node node, int maxSteps, int
131         rootPlayer, int size) {
132         Board board = node.board.copy();
133         int currentPlayer = node.player;
134         int wallsLeft1 = node.walls1;
135         int wallsLeft2 = node.walls2;
136
137         for (int step = 0; step < maxSteps; step++) {
138             if (isWin(board, 1, size)) {
139                 return 1;
140             }
141             if (isWin(board, 2, size)) {
142                 return 2;
143             }
144
145             int walls = currentPlayer == 1 ? wallsLeft1 :
146                 wallsLeft2;
147             List<Move> moves = getMoves(board, currentPlayer,
148                 walls);
149             if (moves.isEmpty()) {
150                 return 3 - currentPlayer;
151             }
152
153             Move best = pickRolloutMove(board, moves,
154                 currentPlayer, rootPlayer, size, wallsLeft1,
155                 wallsLeft2);

```

```

152         applyMove(board, best);
153
154         if (best.getMoveType() == MoveType.PLACE_WALL) {
155             if (currentPlayer == 1) {
156                 --wallsLeft1;
157             } else {
158                 --wallsLeft2;
159             }
160         }
161         currentPlayer = 3 - currentPlayer;
162     }
163
164     return evaluate(board, rootPlayer, size, wallsLeft1,
165                  wallsLeft2) > 0 ? rootPlayer : (3 - rootPlayer);
166 }
167
168 private void backpropagate(Node node, int winner, int
169     playerId) {
170     while (node != null) {
171         ++node.visits;
172         if (winner == playerId) {
173             ++node.wins;
174         }
175         node = node.parent;
176     }
177 }
178
179 private double uct(Node n) {
180     if (n.visits == 0) {
181         return Double.MAX_VALUE;
182     }
183
184     double exploitation = n.wins / n.visits;
185     double exploration = C * Math.sqrt(Math.log(n.parent.
186         visits) / n.visits);
187     double heuristic = normalizedEvaluate(n.board, n.parent
188         .player, n.walls1, n.walls2);
189
190     return exploitation + exploration + heuristic;
191 }

```

```

188
189     private Move pickRolloutMove(Board board, List<Move> moves,
190         int currentPlayer, int rootPlayer, int size, int
191         wallsLeft1, int wallsLeft2) {
192         for (Move move : moves) {
193             if (move.getMoveType() == MoveType.MOVE_PLAYER
194                 && move.getEndPosition().row() ==
195                 getTargetRow(currentPlayer, size)) {
196                 return move;
197             }
198         }
199
200         int walls = currentPlayer == 1 ? wallsLeft1 :
201             wallsLeft2;
202         if (walls > 0 && calculateShortestPath(board,
203             getCurrentPosition(board, 3 - currentPlayer),
204             getTargetRow(3 - currentPlayer, size)) <= 2) {
205             Move tacticalWall = moves.stream()
206                 .filter(move -> move.getMoveType() ==
207                     MoveType.PLACE_WALL)
208                 .max(Comparator.comparingInt(move ->
209                     wallImpact(board, move, currentPlayer,
210                         size)))
211                 .orElse(null);
212             if (tacticalWall != null && wallImpact(board,
213                 tacticalWall, currentPlayer, size) > 0) {
214                 return tacticalWall;
215             }
216         }
217
218         if (random.nextDouble() < 0.22) {
219             return moves.get(random.nextInt(moves.size()));
220         }
221
222         int sampleSize = Math.min(18, moves.size());
223         List<Move> sampled = new ArrayList<>(sampleSize);
224         Set<Integer> usedIndexes = new HashSet<>();
225         while (sampled.size() < sampleSize) {
226             int index = random.nextInt(moves.size());
227             if (usedIndexes.add(index)) {

```

```

219         sampled.add(moves.get(index));
220     }
221 }
222
223     sampled.sort((a, b) -> Integer.compare(
224         scoreRolloutMove(board, b, currentPlayer,
225             rootPlayer, size, wallsLeft1, wallsLeft2),
226         scoreRolloutMove(board, a, currentPlayer,
227             rootPlayer, size, wallsLeft1, wallsLeft2)
228     ));
229     int top = Math.max(1, Math.min(4, sampled.size()));
230     return sampled.get(random.nextInt(top));
231 }
232
233 private int scoreRolloutMove(Board board, Move move, int
234     currentPlayer, int rootPlayer, int size, int wallsLeft1,
235     int wallsLeft2) {
236     Board tmp = board.copy();
237     applyMove(tmp, move);
238     int newWallsLeft1 = wallsLeft1;
239     int newWallsLeft2 = wallsLeft2;
240     if (move.getMoveType() == MoveType.PLACE_WALL) {
241         if (currentPlayer == 1) {
242             --newWallsLeft1;
243         } else {
244             --newWallsLeft2;
245         }
246     }
247
248     int score = evaluate(tmp, rootPlayer, size,
249         newWallsLeft1, newWallsLeft2);
250     if (currentPlayer != rootPlayer) {
251         score = -score;
252     }
253     if (currentPlayer == rootPlayer) {
254         score += movementPreference(move, board,
255             currentPlayer, size, recentPositions);
256     }
257     return score;
258 }

```

```

253
254     private int scoreMove(Board board, Move move, int playerId,
255         int wallsLeft1, int wallsLeft2) {
256         Board copy = board.copy();
257         applyMove(copy, move);
258         int size = (int) Math.sqrt(copy.getTiles().size());
259         int newWallsLeft1 = wallsLeft1;
260         int newWallsLeft2 = wallsLeft2;
261         if (move.getMoveType() == MoveType.PLACE_WALL) {
262             if (move.getPlayerId() == 1) {
263                 --newWallsLeft1;
264             } else {
265                 --newWallsLeft2;
266             }
267         }
268         int preference = playerId == rootPlayerId ?
269             movementPreference(move, board, playerId, size,
270             recentPositions) : 0;
271         return evaluate(copy, playerId, size, newWallsLeft1,
272             newWallsLeft2) + preference;
273     }
274
275     private double normalizedEvaluate(Board board, int playerId
276         , int wallsLeft1, int wallsLeft2) {
277         int size = (int) Math.sqrt(board.getTiles().size());
278         return Math.tanh(evaluate(board, playerId, size,
279             wallsLeft1, wallsLeft2) / 500.0) * 0.15;
280     }
281
282     private double robustChildScore(Node node) {
283         if (node.visits == 0) {
284             return 0;
285         }
286         return node.visits + node.wins / node.visits;
287     }
288
289     private double bestWinRate(Node root) {
290         return root.children.stream().mapToDouble(n -> n.visits
291             == 0 ? 0 : n.wins / n.visits).max().orElse(0);
292     }

```

```

286
287     public boolean isWin(Board board, int playerId, int size) {
288         int row = getCurrentPosition(board, playerId).row();
289         return (playerId == 1 && row == 0) || (playerId == 2 &&
                row == size - 1);
290     }
291
292     static class Node {
293         Board board;
294         Node parent;
295         List<Node> children = new ArrayList<>();
296         List<Move> untriedMoves = new ArrayList<>();
297         Move move;
298
299         int visits = 0;
300         double wins = 0;
301
302         int player;
303         int walls1, walls2;
304
305         Node(Board board, Node parent, Move move, int player,
                int w1, int w2) {
306             this.board = board;
307             this.parent = parent;
308             this.move = move;
309             this.player = player;
310             this.walls1 = w1;
311             this.walls2 = w2;
312         }
313
314         int getWalls() {
315             return player == 1 ? walls1 : walls2;
316         }
317     }
318 }

```

Реализация обучения весов функции оценки

Листинг Г.1 — Класс EvaluationWeightsStore

```

1 public final class EvaluationWeightsStore {
2     private static final Path FILE = Path.of("data", "
        evaluation-weights.properties");
3     private static final Path HISTORY_FILE = Path.of("data", "
        evaluation-weights-history.csv");
4     private static EvaluationWeights current;
5
6     private EvaluationWeightsStore() {}
7
8     public static synchronized EvaluationWeights current() {
9         if (current == null) {
10             current = load();
11         }
12         return current;
13     }
14
15     public static synchronized EvaluationTrainingUpdate
        learnFromGame(List<EvaluationLearningSample> samples,
        int winnerId, String source) {
16         EvaluationWeights beforeWeights = current();
17         if (samples.isEmpty() || winnerId == 0) {
18             return new EvaluationTrainingUpdate(false, winnerId
                , samples.size(), new int[7], beforeWeights,
                beforeWeights);
19         }
20
21         int[] gradient = new int[7];
22         for (EvaluationLearningSample sample : samples) {
23             int reward = sample.playerId() == winnerId ? 1 :
                -1;
24             EvaluationFeatures delta = sample.delta();
25             gradient[0] += reward * delta.pathAdvantage();
26             gradient[1] += reward * delta.myMobility();
27             gradient[2] += reward * delta.enemyMobilityPenalty

```

```

        ();
28         gradient[3] += reward * delta.wallAdvantage();
29         gradient[4] += reward * delta.progressAdvantage();
30         gradient[5] += reward * delta.myEndgame();
31         gradient[6] += reward * delta.enemyEndgamePenalty()
            ;
32     }
33
34     if (isZero(gradient)) {
35         return new EvaluationTrainingUpdate(false, winnerId
            , samples.size(), gradient, beforeWeights,
            beforeWeights);
36     }
37
38     current = beforeWeights.adjustedByGameGradient(gradient
        , 1, samples.size());
39     save(current);
40     appendHistory(source, winnerId, samples.size(),
        gradient, beforeWeights, current);
41     return new EvaluationTrainingUpdate(true, winnerId,
        samples.size(), gradient, beforeWeights, current);
42 }
43
44 private static EvaluationWeights load() {
45     if (!Files.exists(FILE)) {
46         EvaluationWeights defaults = EvaluationWeights.
            defaults();
47         save(defaults);
48         return defaults;
49     }
50
51     Properties properties = new Properties();
52     try (Reader reader = Files.newBufferedReader(FILE,
        StandardCharsets.UTF_8)) {
53         properties.load(reader);
54         return new EvaluationWeights(
55             readInt(properties, "pathAdvantage",
                EvaluationWeights.defaults().
                    pathAdvantage()),
56             readInt(properties, "myMobility",

```

```

        EvaluationWeights.defaults().myMobility
        ()),
57         readInt(properties, "enemyMobility",
            EvaluationWeights.defaults().
            enemyMobility()),
58         readInt(properties, "wallAdvantage",
            EvaluationWeights.defaults().
            wallAdvantage()),
59         readInt(properties, "progressAdvantage",
            EvaluationWeights.defaults().
            progressAdvantage()),
60         readInt(properties, "myEndgame",
            EvaluationWeights.defaults().myEndgame()
            ),
61         readInt(properties, "enemyEndgame",
            EvaluationWeights.defaults().
            enemyEndgame()),
62         readLong(properties, "samples", 0)
63     );
64     } catch (IOException | IllegalArgumentException e) {
65         EvaluationWeights defaults = EvaluationWeights.
            defaults();
66         save(defaults);
67         return defaults;
68     }
69 }
70
71 private static void save(EvaluationWeights weights) {
72     try {
73         Files.createDirectories(FILE.getParent());
74         Files.writeString(FILE, serialize(weights),
            StandardCharsets.UTF_8);
75     } catch (IOException ignored) {
76         // Если файл недоступен, алгоритмы продолжают работать с
            весами в памяти.
77     }
78 }
79
80 private static String serialize(EvaluationWeights weights)
    {

```

```

81         return """
82             pathAdvantage=%d
83             myMobility=%d
84             enemyMobility=%d
85             wallAdvantage=%d
86             progressAdvantage=%d
87             myEndgame=%d
88             enemyEndgame=%d
89             samples=%d
90             """.formatted(
91                 weights.pathAdvantage(),
92                 weights.myMobility(),
93                 weights.enemyMobility(),
94                 weights.wallAdvantage(),
95                 weights.progressAdvantage(),
96                 weights.myEndgame(),
97                 weights.enemyEndgame(),
98                 weights.samples()
99         );
100     }
101
102     private static void appendHistory(String source, int
        winnerId, int sampleCount, int[] gradient,
        EvaluationWeights before, EvaluationWeights after) {
103         try {
104             Files.createDirectories(HISTORY_FILE.getParent());
105             if (!Files.exists(HISTORY_FILE)) {
106                 Files.writeString(HISTORY_FILE, historyHeader()
107                     , StandardCharsets.UTF_8);
108             }
109             Files.writeString(HISTORY_FILE, historyRow(source,
110                 winnerId, sampleCount, gradient, before, after),
111                 StandardCharsets.UTF_8, StandardOpenOption.
112                     APPEND);
113         } catch (IOException ignored) {
114             // История полезна для диплома, но недоступность файла не
115             должна ломать обучение.
116         }
117     }

```

```

115     private static String historyHeader() {
116         return "timestamp,source,winnerId,sampleCount,"
117             + "gradientPath,gradientMyMobility,
                gradientEnemyMobility,gradientWall,
                gradientProgress,gradientMyEndgame,
                gradientEnemyEndgame,"
118             + "beforePath,beforeMyMobility,
                beforeEnemyMobility,beforeWall,
                beforeProgress,beforeMyEndgame,
                beforeEnemyEndgame,beforeSamples,"
119             + "afterPath,afterMyMobility,afterEnemyMobility
                ,afterWall,afterProgress,afterMyEndgame,
                afterEnemyEndgame,afterSamples\n";
120     }
121
122     private static String historyRow(String source, int
        winnerId, int sampleCount, int[] gradient,
        EvaluationWeights before, EvaluationWeights after) {
123         return String.join(", ",
124             LocalDateTime.now().toString(),
125             escape(source),
126             Integer.toString(winnerId),
127             Integer.toString(sampleCount),
128             Integer.toString(gradient[0]),
129             Integer.toString(gradient[1]),
130             Integer.toString(gradient[2]),
131             Integer.toString(gradient[3]),
132             Integer.toString(gradient[4]),
133             Integer.toString(gradient[5]),
134             Integer.toString(gradient[6]),
135             Integer.toString(before.pathAdvantage()),
136             Integer.toString(before.myMobility()),
137             Integer.toString(before.enemyMobility()),
138             Integer.toString(before.wallAdvantage()),
139             Integer.toString(before.progressAdvantage()),
140             Integer.toString(before.myEndgame()),
141             Integer.toString(before.enemyEndgame()),
142             Long.toString(before.samples()),
143             Integer.toString(after.pathAdvantage()),
144             Integer.toString(after.myMobility()),

```

```

145         Integer.toString(after.enemyMobility()),
146         Integer.toString(after.wallAdvantage()),
147         Integer.toString(after.progressAdvantage()),
148         Integer.toString(after.myEndgame()),
149         Integer.toString(after.enemyEndgame()),
150         Long.toString(after.samples())
151     ) + "\n";
152 }
153
154 private static String escape(String value) {
155     String safe = value == null ? "" : value.replace("\\",
156         "\\\"");
157     return "\\\" + safe + "\"";
158 }
159
160 private static int readInt(Properties properties, String
161     key, int fallback) {
162     return Integer.parseInt(properties.getProperty(key,
163         Integer.toString(fallback)));
164 }
165
166 private static long readLong(Properties properties, String
167     key, long fallback) {
168     return Long.parseLong(properties.getProperty(key, Long.
169         toString(fallback)));
170 }
171
172 private static boolean isZero(int[] values) {
173     for (int value : values) {
174         if (value != 0) {
175             return false;
176         }
177     }
178     return true;
179 }
180 }

```